

Plot in a Pot

Um Editor Visual e Simulador para Histórias
Interativas e Lógica Narrativa

Miguel Gomes Silva

Student No. 2221462

Tomás Alexandre dos Santos Reis Lopes

Student No. 2222970

Supervisores: Anabela Gonçalves Rodrigues Marto

*Professora Adjunta, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Miguel Cerdeira Marreiros Negrão

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Nelson Martins Ferreira

*Professor Adjunto, Escola Superior de
Tecnologia e Gestão do Politécnico de Leiria*

Escola Superior de Tecnologia e Gestão

Departamento de Engenharia Informática

Licenciatura em Engenharia Informática

UC de Projeto Informático

Trabalho de Projeto da unidade curricular de Projeto Informático realizado sob a orientação da Professora Doutora Anabela Gonçalves Rodrigues Marto e do Professor Doutor Miguel Cerdeira Marreiros Negrão e do Professor Doutor Nelson Martins Ferreira

Leiria, Junho 2026

Plot in a Pot

Copyright © 2026 - Miguel Gomes Silva, Escola Superior de Tecnologia e Gestão.

A presente dissertação é um trabalho original, elaborado exclusivamente para este fim, tendo sido devidamente citados todos os autores cujos estudos contribuíram para a sua elaboração. É permitida a sua reprodução parcial com indicação do autor e referência ao grau, ano letivo, instituição—*Politécnico de Leiria*—e data da defesa pública.

O presente trabalho beneficiou da utilização do modelo *IPLeiria-Thesis*.

Agradecimentos

Gostaríamos de expressar o nosso mais sincero agradecimento a todos aqueles que, direta ou indiretamente, contribuíram para a realização deste projeto informático e para a elaboração deste relatório.

À nossa orientadora, Professora Doutora Anabela Gonçalves Rodrigues Marto, pelo voto de confiança, paciência, constante incentivo e precioso apoio técnico e científico manifestados ao longo de todas as fases deste trabalho. A sua orientação foi de extrema importância para a superação dos desafios metodológicos e organizacionais.

Aos nossos coorientadores, Professor Doutor Miguel Cerdeira Marreiros Negrão e Professor Doutor Nelson Martins Ferreira, pelas valiosas discussões, sugestões práticas e comentários de revisão rigorosos que ajudaram a elevar a qualidade técnica e arquitetural da nossa solução.

À Escola Superior de Tecnologia e Gestão (ESTG) do Politécnico de Leiria, por disponibilizar os recursos académicos e o ambiente propício ao nosso desenvolvimento pessoal e profissional ao longo da Licenciatura em Engenharia Informática.

Aos participantes que voluntariamente colaboraram na bateria de testes de usabilidade e acessibilidade, cujo feedback construtivo e sincero foi inestimável para a validação e refinamento prático do software.

Por fim, expressamos um agradecimento muito especial às nossas famílias e amigos, pelo apoio incondicional, incentivo constante e paciência demonstrados durante este percurso académico.

Resumo

O desenvolvimento de ficção interativa e jogos baseados em escolhas enfrenta sérios desafios de usabilidade e integridade estrutural à medida que a ramificação narrativa cresce. Embora existam ferramentas populares de autoria como o Twine, a maioria carece de suporte nativo integrado para gestão de localização multi-idioma e de motores de validação estática de caminhos lógicos para detetar inconsistências (como becos sem saída e cenas órfãs) antes da fase de compilação ou execução. O presente trabalho de projeto propõe o *Plot in a Pot* (PiaP), um editor visual e simulador baseado em grafos que unifica a autoria de narrativas escritas no formato aberto Tweep 3 (SugarCube) com um motor integrado de tradução de textos e validação estática de caminhos.

Implementado em React e no ecossistema React Flow, o PiaP destaca-se pela sua estética neo-brutalista de alto contraste visual, concebida para mitigar barreiras de acessibilidade cognitiva e cromática para autores com daltonismo severo (como deuteranopia e protanopia). O sistema integra: (i) um parser síncrono bidirecional para ler e reconstruir topologias Tweep 3 em tempo real; (ii) uma matriz de localização baseada em grelha bidimensional com importação e exportação de CSV; (iii) um motor de validação estática suportado por algoritmos de Pesquisa em Largura (BFS) para analisar a acessibilidade de cenas; (iv) um modo de simulação interativo dotado de um agente inteligente autónomo (*Smart Walker*) que utiliza procura por novidade (*Novelty Search*) para percorrer e testar caminhos automaticamente; e (v) arestas com rota curva parametrizada por splines cúbicas de Catmull-Rom.

A avaliação qualitativa e quantitativa do sistema foi efetuada através de testes com seis utilizadores com diferentes perfis técnicos e de acessibilidade. Os resultados revelaram uma taxa média de conclusão de tarefas de 91,7% e uma elevada usabilidade e satisfação, validando a eficácia do *Plot in a Pot* como uma ferramenta viável e robusta para autores e *game designers* na prototipagem e desenvolvimento de narrativas interativas complexas.

Palavras-Chave: Ficção Interativa, Editores de Grafos, Localização de Jogos, Acessibilidade Visual, Validação Estática de Fluxo, Twine, SugarCube.

Abstract

The development of interactive fiction and choice-based games faces serious challenges in usability and structural integrity as the narrative branching expands. Although popular authoring tools like Twine exist, most lack integrated native support for multi-language localization management and static path-validation engines to detect inconsistencies (such as dead ends and orphan scenes) before compilation or execution. This project work proposes *Plot in a Pot* (PiaP), a graph-based visual editor and simulator that unifies the authoring of interactive stories written in the open Twee 3 format (SugarCube) with an integrated translation matrix and static path-validation engine.

Implemented in React and using the React Flow ecosystem, PiaP stands out for its high-contrast neo-brutalist visual style, specifically designed to mitigate cognitive and chromatic accessibility barriers for authors with severe color blindness (such as deuteranopia and protanopia). The system features: (i) a synchronous bidirectional parser to read and reconstruct Twee 3 topologies in real time; (ii) a translation matrix layout with CSV import and export capabilities; (iii) a static validation engine powered by Breadth-First Search (BFS) algorithms to audit scene reachability; (iv) an interactive simulation mode equipped with an autonomous agent (*Smart Walker*) that leverages novelty search heuristics to automatically test paths; and (v) customizable edge routing parameterized by Catmull-Rom cubic splines.

Qualitative and quantitative evaluation was conducted through user testing with six participants of varying technical backgrounds and accessibility needs. The results showed a 91.7% average task completion rate alongside high levels of usability and satisfaction, validating the effectiveness of *Plot in a Pot* as a viable and robust tool for authors and game designers during the prototyping and development of complex interactive narratives.

Keywords: Interactive Fiction, Graph Editors, Game Localization, Visual Accessibility, Static Flow Validation, Twine, SugarCube.

Declaração sobre o Uso de Inteligência Artificial

i Orientação Para a Escrita

É uma boa prática académica indicar brevemente como, porquê e quando a IA foi utilizada. Explicar as ações tomadas e as razões que lhes estão subjacentes promove a reflexão crítica, aprofunda a compreensão do papel da IA na aprendizagem e ajuda a avaliar o seu impacto na qualidade e integridade do trabalho. Esta página pode ser removida definindo a opção `aiack` como `false` nas opções da classe do documento.

i Exemplo de Utilização

Reconheço a utilização do Gemini (<https://gemini.google.com>) para aperfeiçoar o tom académico e melhorar a precisão linguística deste trabalho, incluindo aspectos de gramática, pontuação e vocabulário.

Descrição da Utilização

Instrução 1: Num parágrafo, descreva simplesmente o que é o LaTeX.

Resposta 1: O LaTeX é um sistema de composição de alta qualidade, normalmente utilizado para produzir documentos científicos e matemáticos, devido ao seu poderoso manuseamento de fórmulas e bibliografias. Ao contrário dos processadores de texto, utiliza ficheiros de texto simples com etiquetas de marcação para definir a estrutura e a formatação de um documento, permitindo um controlo preciso da disposição e do aspeto. É especialmente popular no meio académico e na investigação para a criação de teses, artigos de revistas e apresentações.

Conteúdo

<i>Lista de Figuras</i>	xiv
<i>Lista de Tabelas</i>	xvi
<i>Siglas</i>	xviii
1 Introdução	1
1.1 Objeto do Trabalho	1
1.2 Justificação e Pertinência do Tema	1
1.3 Objetivos do Trabalho	2
1.3.1 Objetivo Geral	2
1.3.2 Objetivos Específicos	2
1.4 Requisitos Gerais do Sistema	3
1.5 Métodos e Técnicas Utilizadas	3
1.6 Estrutura do Documento	4
2 Fundamentação Teórica e Estado da Arte	5
2.1 Contexto Teórico	5
2.2 Teoria de Grafos na Narrativa Interativa	6
2.2.1 Tipos de Grafos e Aplicação Narrativa	6
2.2.2 Grafos Direcionados Acíclicos (Grafo Direcionado Acíclico (Directed Acyclic Graph)s (DAGs)) vs. Grafos com Ciclos	7
2.2.3 Análise Matemática da Explosão de Complexidade Lógica	7
2.3 Software de Criação de Histórias não Lineares	8
2.3.1 Twine	9
2.3.2 Ink (Inkle Studios)	9
2.3.3 ChoiceScript	9
2.3.4 Yarn Spinner	10
2.3.5 Narrative Flow	10
2.3.6 Articy Draft	10
2.3.7 Análise Comparativa	10
2.4 Integração de Modelos de Linguagem de Grande Escala (LLMs) Locais . .	12
2.4.1 Análise de Famílias de Modelos e Recomendações	13

3	Conceção e Implementação	14
3.1	Arquitetura Geral do Sistema	14
3.2	Requisitos Funcionais e Não Funcionais	15
3.2.1	Requisitos Funcionais	15
3.2.2	Requisitos Não Funcionais	16
3.3	Enquadramento e Implementação Tecnológica	17
3.3.1	Justificação das Decisões de Design	17
3.4	Modelação de Dados: Lista de Adjacência Bidirecional	18
3.5	Parser Síncrono e Especificação Twee 3	19
3.5.1	Importação e Conversão de Coordenadas em Zonas	19
3.5.2	Passagens Especiais	20
3.5.3	Modelo SugarCube (Ligações e Macros)	20
3.6	Interação Visual, Edição e Acessibilidade	20
3.6.1	Zonas de Agrupamento Visual	20
3.6.2	Arestas Curvadas com Waypoints	21
3.6.3	Gestão de Variáveis Estilo Figma	22
3.6.4	Acessibilidade para Autores com Dificuldades Visuais	22
3.6.5	Modelação de Dados e Sincronização com o React Flow	23
3.7	Validação Estática e Simulação	24
3.7.1	Algoritmo de Validação Estática e Exploração do Espaço de Estados (Pesquisa em Largura (Breadth-First Search) (BFS))	24
3.7.2	Simulador em Tempo Real (PlayMode) e Smart Walker	26
3.7.3	Motor de Compilação JIT de Macros SugarCube	27
3.8	Integração de Modelos de Linguagem (LLMs)	27
3.8.1	Modelo de Integração Local sem Custos	28
3.8.2	Papel do LLM: Conversão e Importação Automática de Histórias	28
4	Desenvolvimento e Avaliação	30
4.1	Cronograma e Fases de Desenvolvimento	30
4.2	Estrutura do Código e Detalhes de Implementação	31
4.2.1	Componentes de Interface (src/components/)	32
4.2.2	Módulos de Lógica e Utilitários (src/utils/)	40
4.2.3	Fluxo de Dados e Reatividade	43
4.3	Testes de Conversão e Comparação de Modelos de Linguagem (IA Local)	44
4.3.1	Primeira Fase: Testes em Máquina Local (RTX 3050)	44
4.3.2	Segunda Fase: Testes com Maior Capacidade de Processamento	44
4.3.3	Identificação do Bug Técnico no Ollama	45
4.3.4	Estado Atual da Funcionalidade (Beta) e Trabalho Futuro	45
4.4	Análise de Desempenho e Complexidade Algorítmica	45
4.4.1	Complexidade Algorítmica Teórica	46
4.4.2	Benchmarks de Desempenho Empíricos	46
4.5	Metodologia dos Testes de Utilizador	47

4.5.1	Perfil dos Participantes	47
4.5.2	Tarefas Avaliadas	48
4.6	Resultados dos Testes e Desenvolvimento Iterativo	48
4.6.1	Primeira Leva: Protótipo Inicial (Abril de 2026)	48
4.6.2	Segunda Leva: Protótipo Intermédio (Maio de 2026)	49
4.6.3	Terceira Leva: Protótipo Final (Junho de 2026)	49
4.6.4	Feedback Qualitativo e Usabilidade Visual	51
5	Conclusão e Trabalho Futuro	52
5.1	Conclusão	52
5.2	Trabalho Futuro	53
	<i>Bibliography</i>	56
	Apêndices	
A	Manual de Utilização e Instalação	61
A.1	Requisitos do Sistema e Instalação Local	61
A.1.1	Instalação do Editor Web	61
A.1.2	Configuração do Servidor de IA Local (Ollama)	62
A.2	Manual de Utilização do Editor Gráfico	62
A.2.1	O Canvas Interativo	62
A.2.2	Criação e Edição de Nós Narrativos (Cenas)	63
A.2.3	Criação de Ligações e Arestas de Escolha	63
A.2.4	Agrupamento em Zonas (Capítulos)	63
A.3	Programação de Lógica Narrativa e Variáveis	64
A.3.1	Criação de Variáveis (Figma-Style Tokens)	64
A.3.2	Edição de Lógica condicional por Código	64
A.3.3	Editor de Lógica Estruturada	65
A.4	Tradução e Localização Multi-idioma	65
A.4.1	Configurar Idiomas	65
A.4.2	Editar Textos na Matriz	65
A.4.3	Importação e Exportação de Ficheiros Comma-Separated Values (CSV)	65
A.5	Validação Lógica e Simulação de Jogo	66
A.5.1	O Painel de Erros de Validação Estática	66
A.5.2	Simulação Ativa (PlayMode) e Testes com o Smart Walker	66
A.6	Operações de Ficheiros e Publicação do Projeto	66
A.6.1	Importar Ficheiros Twee 3	66
A.6.2	Exportar e Compilar a História	67

B	Guia de Sintaxe e Modelagem de Macros Lógicas	68
B.1	Estrutura de um Documento Twee 3	68
B.1.1	Passagens Especiais de Sistema	68
B.1.2	Passagens Narrativas (Cenas)	69
B.2	Sintaxe de Macros Lógicas (SugarCube)	69
B.2.1	Macro de Atribuição: <<set>>	69
B.2.2	Macro de Controlo Condicional: <<if>>	69
B.3	Padrões Comuns de Modelação Narrativa	70
B.3.1	Padrão 1: Inventário e Portas Trancadas	70
B.3.2	Padrão 2: Contador de Tentativas e Fim de Jogo	70
B.3.3	Padrão 3: Caminhos Ocultos e Nós Secretos	71
 Anexos		
L	Especificação Técnica Normativa do Formato Twee 3	75
L.1	Sintaxe Geral e Passagens	75
L.1.1	Sintaxe do Cabeçalho da Passagem	75
L.2	Especificação de Metadados de Posicionamento	76
L.2.1	Campos de Geometria JavaScript Object Notation (JSON)	76
L.3	Especificação de Hiperligações e Transições	76

Lista de Figuras

3.1	Diagrama de blocos detalhado da arquitetura conceptual e fluxo de dados. . .	15
4.1	Interface gráfica do canvas de grafos do Plot in a Pot, demonstrando a estética neo-brutalista de alto contraste com nós de cena e agrupamentos visuais (Zonas).	33
4.2	Ecrã da sandbox de simulação (PlayMode), ilustrando a execução em tempo real da história com o painel de depuração de variáveis lógicas.	37
4.3	Diálogo de gestão de variáveis globais (VariablesModal), apresentando os tokens de dados e a renomeação segura baseada em Expressões Regulares (Regex).	38
4.4	Matriz de localização integrada no editor, exibindo a grelha de chaves de texto e a respetiva tradução direta para os idiomas selecionados.	40
4.5	Interface do editor visual de lógica estruturada (VisualBlocksEditor), dividindo as passagens em painéis de prosa, setters de variáveis e escolhas condicionais.	41

Lista de Tabelas

2.1	Análise comparativa de motores de ficção interativa.	11
3.1	Mapeamento de operadores lógicos SugarCube para JavaScript.	27
4.1	Cronograma detalhado das fases de desenvolvimento do projeto.	30
4.2	Complexidade algorítmica teórica dos subsistemas nucleares.	46
4.3	Resultados empíricos de benchmarks de desempenho (em milissegundos). . .	47
4.4	Taxa de sucesso e tempo de conclusão (em minutos) das tarefas por utilizador (Leva Final).	50
4.5	Pontuação obtida no questionário System Usability Scale (SUS) na leva final.	51

Siglas

API	Application Programming Interface
AST	Árvore de Sintaxe Abstrata (<i>Abstract Syntax Tree</i>)
BFS	Pesquisa em Largura (<i>Breadth-First Search</i>)
CSV	Comma-Separated Values
DAG	Grafo Direcionado Acíclico (<i>Directed Acyclic Graph</i>)
DFS	Pesquisa em Profundidade (<i>Depth-First Search</i>)
ESTG	Escola Superior de Tecnologia e Gestão
FI	Ficção Interativa
IFTF	Interactive Fiction Technology Foundation
JSON	JavaScript Object Notation
PiaP	Plot in a Pot
Regex	Expressões Regulares
SPA	Single Page Application

1

Introdução

O desenvolvimento de jogos e narrativas interativas tem vindo a ganhar uma importância crescente no panorama do entretenimento e da educação. Diferente do cinema ou da literatura tradicional, a **Ficção Interativa (FI)** coloca o utilizador no centro da tomada de decisões, permitindo-lhe moldar o rumo dos acontecimentos através das suas escolhas. Esta não-linearidade narrativa, no entanto, introduz uma complexidade acrescida na escrita e no planeamento das histórias, exigindo ferramentas capazes de gerir fluxos de informação complexos e lógica de estados dinâmicos.

Neste contexto surge o *Plot in a Pot*, um editor visual e simulador de narrativas interativas. O projeto foi concebido para resolver os principais desafios enfrentados pelos autores de histórias não-lineares, combinando uma interface visual de alto desempenho baseada em grafos direcionados com um motor de simulação que suporta a execução de código e lógica em tempo real.

1.1 Objeto do Trabalho

O objeto deste projeto consiste no desenvolvimento do **Plot in a Pot (PiaP)**, uma aplicação e ambiente de desenvolvimento concebido para a criação, estruturação, simulação e tradução de narrativas interativas e não-lineares. O sistema foca-se na integração e suporte nativo ao ecossistema do motor Twine, operando especificamente com o formato de ficheiros Tweep (versão 3) e com a macro-linguagem SugarCube. Através de uma interface gráfica baseada em grafos, a ferramenta permite a autores e *game designers* gerir ramificações complexas e bases de dados de localização de uma forma unificada.

1.2 Justificação e Pertinência do Tema

O *design* de narrativas para jogos não-lineares enfrenta frequentemente problemas severos de escala, onde árvores de diálogo e escolhas crescem exponencialmente, tornando-se difíceis de rastrear e propensas a erros lógicos. Embora o Twine seja uma das ferramentas de referência da comunidade independente e académica (**Interactive Fiction Technology**

Foundation, 2026) para a prototipagem rápida de FI, o desenvolvimento de projetos profissionais de grande escala é severamente limitado pela falta de ferramentas nativas para a gestão multi-idioma (localização) e pela ausência de validadores estáticos de fluxo que previnam caminhos mortos antes da compilação.

O PiaP justifica-se ao resolver esta lacuna, introduzindo uma estética neo-brutalista de alto contraste que otimiza o espaço de trabalho e melhora a acessibilidade de autores com limitações de visão (Babich, 2023; Loranger, 2022), permitindo isolar cadeias de texto rígidas (*hardcoded*) através de um sistema abstrato de chaves de tradução dinâmicas e oferecendo segurança matemática no mapeamento de rotas narrativas.

1.3 Objetivos do Trabalho

1.3.1 Objetivo Geral

Desenvolver uma plataforma *web* modular que unifique a edição visual de grafos de histórias interativas Tweep com um motor (*engine*) integrado de localização multi-idioma e validação de caminhos lógicos.

1.3.2 Objetivos Específicos

Para a concretização do objetivo geral, definiram-se os seguintes objetivos específicos:

1. **Interface Visual Neo-Brutalista e Acessível:** Implementar um *canvas* de grafos interativo e de alto desempenho no estilo neo-brutalista com forte contraste visual, que suporte a criação de nós de cena, zonas de agrupamento visual, *scripts* (JavaScript) e folhas de estilo (CSS), facilitando a utilização por pessoas com daltonismo.
2. **Otimização Visual com Waypoints:** Desenvolver um algoritmo de rota iterativo através de *waypoints* geométricos nas arestas para otimização visual e organização do espaço de trabalho.
3. **Parser Bidirecional Tweep3:** Construir um analisador (*parser*) bidirecional síncrono em tempo real capaz de ler a sintaxe Twine ([[LinkDestino]]) e reconstruir a topologia do grafo instantaneamente, mantendo a compatibilidade e convertendo coordenadas.
4. **Matriz de Localização Integrada:** Criar uma matriz de localização baseada em grelha *Flexbox* com suporte para importação/exportação em formato CSV e tradução *inline* por células.
5. **Validador de Fluxo baseado em Listas de Adjacência:** Desenvolver um motor de verificação estática baseado em listas de adjacência para detetar nós inacessíveis/órfãos, becos sem saída ou *loops* infinitos na narrativa.
6. **Simulador Imersivo e Tutorial Jogável:** Implementar um ambiente de simulação em tempo real (*PlayMode*) e um tutorial jogável (*Playable Tutorial*) para validação pedagógica das regras do editor.

1.4 Requisitos Gerais do Sistema

Com o intuito de delimitar o âmbito do projeto e as necessidades fundamentais a que o **PiaP** responde, foram estabelecidos os seguintes requisitos gerais que guiam o comportamento e a utilidade da aplicação:

- **Modelador Narrativo Visual:** A aplicação deve disponibilizar uma interface gráfica baseada em grafos que permita modelar e ligar de forma dinâmica as cenas da história.
- **Motor de Tradução Integrado:** Deve fornecer um sistema para traduzir e localizar textos diretamente na ferramenta de autoria, eliminando a dispersão de ficheiros externos.
- **Análise e Validação de Consistência:** O sistema deve auditar as ligações criadas para detetar inconsistências como becos sem saída, nós isolados ou ciclos sem saída apropriada.
- **Compatibilidade com o Ecossistema Twine:** O software deve garantir interoperabilidade bidirecional com o formato padrão Twine 3 (SugarCube), permitindo importar e exportar ficheiros sem perda de metadados.
- **Simulação em Tempo Real:** Fornecer um modo interativo para jogar, depurar e testar a narrativa diretamente na ferramenta sem necessidade de compilação externa.
- **Conversão Inteligente de Texto via IA:** O sistema deve permitir que autores escrevam histórias corridas em linguagem natural e convertam esses textos em grafos estruturados no formato Twine automaticamente usando inteligência artificial local.

1.5 Métodos e Técnicas Utilizadas

O desenvolvimento do projeto apoiou-se em metodologias ágeis com ciclos rápidos de prototipagem e testes de esforço (*stress*) de dados. Ao nível tecnológico e arquitetural, foram utilizadas as seguintes ferramentas e conceitos:

- **Frontend e Canvas de Grafos:** Desenvolvimento baseado na biblioteca React (Meta Open Source, 2013). A escolha justifica-se pela sua elevada popularidade e suporte na indústria (Statista Research Department, 2025) e, fundamentalmente, pela familiaridade e experiência prévia da equipa de desenvolvimento com este ecossistema, o que contribuiu para mitigar a curva de aprendizagem e acelerar a entrega do projeto. A manipulação interativa do grafo e renderização visual foram suportadas pelo ecossistema React Flow (webkid.io, 2021), selecionado pela sua integração direta com a arquitetura baseada em componentes do React.
- **Algoritmia e Teoria de Grafos:** Utilização de estruturas de dados baseadas em listas de adjacências para a implementação de algoritmos de pesquisa (BFS e Pesquisa em Profundidade (Depth-First Search) (DFS)) que validam de forma

estática e iterativa a integridade dos caminhos e simulam estados de variáveis em memória.

- **Internacionalização e Parsing:** Integração da biblioteca `i18next` em conjunto com algoritmos de substituição baseados em `Regexs` para intercetar macros Sugar-Cube e injetar as chaves da base de dados de localização em tempo real.

1.6 Estrutura do Documento

Este relatório está estruturado em cinco capítulos principais. Além desta introdução, o documento apresenta:

- **Capítulo 2: Fundamentação Teórica e Estado da Arte**, onde se efetua uma análise comparativa das ferramentas existentes no mercado e se descrevem os conceitos matemáticos de teoria de grafos aplicados à narrativa.
- **Capítulo 3: Conceção e Implementação**, detalhando a arquitetura do software, a especificação `Twee3`, o modelo de lista de adjacências bidirecional, os modos de visualização e acessibilidade, e a simulação lógica.
- **Capítulo 4: Desenvolvimento e Avaliação**, que descreve o cronograma de desenvolvimento, a estrutura física do código e detalhes de implementação da aplicação, bem como a metodologia e resultados dos testes de utilizador realizados para validar o sistema.
- **Capítulo 5: Conclusão e Trabalho Futuro**, onde são sumariados os resultados alcançados, as limitações da implementação e as propostas para futuros desenvolvimentos.

2

Fundamentação Teórica e Estado da Arte

Neste capítulo são expostos os fundamentos teóricos relativos à modelação matemática de fluxos narrativos interativos, bem como a análise do estado da arte sobre as ferramentas de autoria existentes, destacando os seus pontos fortes e limitações.

2.1 Contexto Teórico

A criação de narrativas não lineares enfrenta frequentemente a “Explosão de Complexidade”, na qual os modelos tradicionais baseados em sequências ou cadeias falham em produzir narrativas logicamente consistentes, resultando muitas vezes em conteúdo repetitivo ou contradições causais (Chen et al., 2021). A poluição visual nas ferramentas tradicionais surge porque fluxogramas simples ou cadeias de eventos não conseguem capturar de forma eficaz as ligações densas e multidimensionais entre batidas narrativas, conduzindo a problemas de “alucinação” em sistemas automatizados, nos quais os eventos previstos carecem de precisão ou de completude (Li et al., 2018). Sem uma estrutura formal, os designers enfrentam dificuldades significativas na gestão de percursos ramificados, o que pode conduzir a evoluções narrativas incoerentes que são cognitivamente exigentes de depurar ou verificar manualmente (Mormoris, 2024).

Para ultrapassar estas limitações estruturais, *frameworks* recentes recorrem a *Narrative Event Evolutionary Graphs* (NEEG) e a *Event-centric Temporal Knowledge Graphs*, de forma a fornecer uma representação estruturada da evolução dos eventos (Li et al., 2018; Yan et al., 2023). Ao tratar as histórias como grafos direcionados com relações temporais e causais explícitas, estes sistemas conseguem garantir um fluxo lógico e prevenir as inconsistências encontradas em designs manuais tipo “esparguete” (Chen et al., 2021; Li et al., 2018; Tang et al., 2022). A implementação de uma *framework* deste tipo permite a utilização de conjuntos de exclusão mútua e métricas de coerência para assegurar que os eventos simultâneos são lógicos e que cada batida narrativa permanece alcançável dentro da hierarquia pretendida (Chen et al., 2021). Esta mudança para uma

organização baseada em grafos transforma as ferramentas narrativas de meras aplicações de desenho em validadores lógicos ativos, garantindo consistência global através das propriedades matemáticas formalizadas da interação entre eventos (Chen et al., 2021; Tang et al., 2022).

Tendo em conta o facto de ainda não existirem normas estabelecidas, ao formalizar uma narrativa como um Conjunto Parcialmente Ordenado (Poset)¹, este projeto pode ir além da simples visualização para criar um Validador Lógico que assegura a integridade estrutural ao nível de arquitetura.

2.2 Teoria de Grafos na Narrativa Interativa

A estrutura básica de uma história interativa assenta na modelação geométrica e lógica de caminhos narrativos. Um fluxo de história pode ser formalmente representado como um Grafo $G = (V, E)$ (Bondy et al., 2008), onde:

- V representa o conjunto de **Vértices** (ou nós), que correspondem a cenas, passagens de texto ou estados lógicos da história.
- E representa o conjunto de **Arestas** (ou ligações direcionadas), que representam as escolhas disponibilizadas ao leitor para transitar de uma cena para a outra.

2.2.1 Tipos de Grafos e Aplicação Narrativa

Dependendo da natureza do jogo ou da história, utilizam-se diferentes tipos de estruturas de grafos:

1. **Grafo Direcionado (Directed Graph):** Uma estrutura onde as ligações têm direção, ou seja, a existência de um caminho de $A \rightarrow B$ não implica o retorno de $B \rightarrow A$. Na matriz de adjacência, isto traduz-se numa estrutura não-simétrica:

$$M[i][j] \neq M[j][i]$$

Representa o fluxo narrativo puro, onde cada escolha conduz o leitor numa direção específica.

2. **Grafo Não-Direcionado (Undirected Graph):** Representa relações simétricas e bidirecionais ($A - B$). Embora seja menos comum em narrativa linear, é útil para modelar redes de relacionamento social entre personagens ou conexões espaciais de livre exploração (ex: salas adjacentes de uma masmorra). Na matriz de adjacência, a estrutura é simétrica:

$$M[i][j] = M[j][i]$$

3. **Grafo de Conhecimento (Knowledge Graph):** Modela relações semânticas complexas entre entidades (ex: “personagem conhece objeto”, “porta depende de

¹ As propriedades de reticulado são fundamentais para garantir a integridade lógica da narrativa.

chave”). Exige múltiplas matrizes de adjacência ou uma estrutura tridimensional:

$$M_{relation}[i][j]$$

É ideal para jogos com inventários complexos e decisões baseadas em conhecimento indireto acumulado pelo utilizador.

4. **Grafo Ponderado (Weighted Graph):** Atribui valores (pesos) às arestas:

$$M[i][j] = \text{peso}$$

Estes pesos podem representar custos de recursos (tempo, pontos de vida), custos emocionais do personagem, ou a probabilidade de uma escolha ter sucesso.

2.2.2 Grafos Direcionados Acíclicos (DAGs) vs. Grafos com Ciclos

Tradicionalmente, muitas plataformas limitam a escrita a Grafos Direcionados Acíclicos (*Directed Acyclic Graphs* - **DAGs**). A propriedade acíclica garante que é impossível iniciar num vértice V_i e, através de arestas direcionadas, regressar a V_i . Isto garante um fluxo estritamente unidirecional e progressivo de causa e efeito, facilitando a aplicação de algoritmos como a Ordenação Topológica (*Topological Sorting*) com complexidade linear:

$$O(V + E)$$

A ordenação topológica (através de abordagens como o algoritmo de Kahn ([Kahn, 1962](#))) alinha os vertices linearmente, facilitando o cálculo imediato do progresso da história e a identificação do caminho mais rápido.

No entanto, em narrativas complexas, a imposição de **DAGs** é altamente limitadora. A impossibilidade de introduzir ciclos impede a criação de:

- **Estruturas de Cubo (Hubs):** Onde o jogador explora vários caminhos secundários a partir de uma praça ou menu central e regressa recorrentemente ao mesmo ponto.
- **Mecânicas de Simulação e Tempo:** Eventos baseados em estados lógicos recorrentes, como “aguardar um turno” no mesmo local para fazer avançar variáveis temporais.

Por esta razão, o *Plot in a Pot* adota um modelo de **Grafo Direcionado Geral** que permite ciclos lógicos. Isto requer algoritmos de validação dinâmica em segundo plano (como pesquisas **BFS** e **DFS** ([Cormen et al., 2009](#))) para evitar a ocorrência de loops infinitos sem condições de saída logicamente definidas.

2.2.3 Análise Matemática da Explosão de Complexidade Lógica

A modelação matemática de narrativas interativas permite compreender a necessidade de validadores automatizados ao expor o problema clássico da explosão de estados. Seja

$G = (V, E)$ o grafo que modela a história. Se assumirmos uma estrutura de ramificação pura sem ciclos (um grafo em árvore direcionado), com um fator médio de ramificação b (número médio de escolhas por nó) e uma profundidade média da narrativa d (número de passagens percorridas do início ao fim), o número total de caminhos narrativos únicos e independentes P cresce de forma exponencial:

$$P = O(b^d)$$

Este crescimento exponencial (também conhecido por explosão combinatória ou combinatoria de escolhas) torna impossível a um autor auditar manualmente todas as combinações de escolhas possíveis à medida que a história se expande.

O problema agrava-se substancialmente com a introdução de ciclos (onde G deixa de ser acíclico, permitindo que $V_i \rightarrow \dots \rightarrow V_i$), uma vez que o número de caminhos matematicamente possíveis passa a ser infinito:

$$P = \infty$$

Neste cenário, a coerência da simulação passa a depender inteiramente da valoração do vetor de variáveis de estado σ . Definimos o espaço de estados global $\mathcal{S}$ da narrativa como o produto cartesiano entre o conjunto de nós de escrita $\mathcal{N}$ e o conjunto de valorações possíveis das variáveis globais da história. Se considerarmos um conjunto de $|\mathcal{V}|$ variáveis de estado do tipo booleanas (por exemplo, registos de inventário como `$tem_chave` ou escolhas morais binárias), o tamanho máximo do espaço de estados explorável $\mathcal{S}$ é dado por:

$$|\mathcal{S}| = |\mathcal{N}| \times 2^{|\mathcal{V}|}$$

Caso as variáveis tomem valores em domínios numéricos maiores (e.g. inteiros para pontos de vida ou reputação), o espaço cresce de forma ainda mais severa. A introdução de loops infinitos que incrementam repetidamente uma variável sem uma condição de paragem adequada fará com que o espaço de estados se torne teoricamente infinito, o que levaria a que uma travessia cega de pesquisa de caminhos nunca terminasse.

Esta propriedade matemática demonstra que a validação estrutural simples de grafos geométricos (baseada puramente na conectividade de vértices) é insuficiente em **FI** com lógica condicional. Justifica-se assim o desenvolvimento de um motor de validação estática capaz de explorar o espaço de estados reais $\langle v, \sigma \rangle$ e implementar podas algorítmicas rigorosas para garantir a paragem do algoritmo de verificação.

2.3 Software de Criação de Histórias não Lineares

O conceito de **FI** refere-se a narrativas estruturadas onde a progressão do texto não é linear, dependendo diretamente das escolhas e ações submetidas pelo utilizador para navegar caminhos alternativos (Montfort, 2005). Com a expansão do desenvolvimento de videojogos independentes e o crescente estudo de narratologia computacional, diversas

ferramentas de autoria emergiram para suportar a criação deste tipo de estruturas ramificadas, destacando-se o Twine, o Ink e o ChoiceScript ([Interactive Fiction Technology Foundation, 2026](#)).

Apresenta-se, de seguida, uma análise detalhada das principais referências do mercado académico e profissional.

2.3.1 Twine

Lançado em 2009 por Chris Klimas ([Klimas, 2009](#)), o Twine é a ferramenta mais popular na comunidade de **FI** baseada em texto.

- **Vantagens:** Extremamente acessível (sem barreira de programação para escrita básica); visualização clara baseada em nós gráficos no canvas; open-source e gratuito (distribuído sob a licença GPL v3, permitindo qualquer utilização, incluindo comercial, sem restrições); rápido para prototipagem rápida.
- **Limitações:** A interface visual torna-se desorganizada e lenta em projetos muito grandes (efeito “prato de esparguete”); a lógica de estado nativa é limitada; a integração com motores de jogos externos (como Unity ou Unreal) é complexa.

O fluxo de trabalho central do Twine assenta na criação de passagens (segmentos individuais da história) e na sua ligação através de hiperligações que representam escolhas do jogador. Estas passagens são exibidas como caixas num canvas com setas a indicar os caminhos narrativos ([Klimas, 2009](#)). Este paradigma mapeia-se diretamente para a representação em canvas de grafos interativos que constitui o núcleo visual do **PiaP**, tornando a correspondência entre o modelo de dados interno e a interface imediata e semanticamente coerente.

2.3.2 Ink (Inkle Studios)

O Ink ([Inkle Studios, 2016](#)) é uma linguagem textual de scripting criada pela Inkle Studios (criadores de jogos como *80 Days*).

- **Vantagens:** Altamente poderosa para controlo fino de estados narrativos, condicionais complexas e ramificações complexas; excelente integração técnica com motores de jogo; licenciada sob a licença MIT (totalmente livre); escala muito bem para projetos de grande envergadura.
- **Limitações:** Não é uma ferramenta visual (opera de forma textual como scripting acoplado a motores de jogo), o que dificulta o planeamento geográfico e a deteção de redundâncias de caminhos por autores não-técnicos; exige conhecimentos de lógica de programação.

2.3.3 ChoiceScript

Desenvolvido pela Choice of Games, o ChoiceScript é uma linguagem de programação simples e baseada em texto para a escrita de romances interativos baseados em escolhas.

- **Vantagens:** Sintaxe simples e linear de fácil compreensão para escritores; possui excelentes ferramentas de teste automático (como `quicktest` e `randomtest`) que facilitam a validação de caminhos e a robustez lógica de romances longos.
- **Limitações:** Não é uma ferramenta visual; o formato de ficheiro é proprietário; a sua licença (CSL) proíbe explicitamente a utilização comercial sem autorização prévia e acordo de distribuição comercial com a Choice of Games ([Choice of Games, 2026](#)), limitando a autonomia de autores independentes.

2.3.4 Yarn Spinner

O Yarn Spinner ([Secret Lab, 2015](#)) é uma ferramenta focada na integração de diálogos em jogos interativos com motor gráfico.

- **Vantagens:** Sintaxe simples (semelhante a scripts de teatro); suporte nativo para variáveis; excelente portabilidade direta para Unity e Unreal Engine.
- **Limitações:** Carece de um planeador visual abstrato robusto fora do motor de jogo principal; não é adequado para desenhar fluxogramas abstratos de histórias antes da fase de produção do jogo.

2.3.5 Narrative Flow

O Narrative Flow ([NarrativeFlow, 2023](#)) é uma ferramenta emergente que tenta colmatar a lacuna entre Twine e Ink.

- **Vantagens:** Oferece melhor organização estrutural do que o Twine e uma visualização mais clara que o Ink; ajuda a gerir a consistência da escrita.
- **Limitações:** É um software pago, com menor maturidade técnica e uma comunidade de utilizadores consideravelmente mais reduzida.

2.3.6 Articy Draft

O Articy Draft ([Articy Software, 2010](#)) é a solução comercial líder da indústria de videogames AAA para planeamento de narrativas e bases de dados de escrita.

- **Vantagens:** Altamente estruturado e profissional; gestão integrada de personagens, inventários e diálogos; escala com eficácia para equipas de desenvolvimento de grandes dimensões.
- **Limitações:** Curva de aprendizagem acentuada; software dispendioso (licença comercial paga); excessivamente complexo e pesado para autores independentes ou projetos pequenos.

2.3.7 Análise Comparativa

A fim de estruturar a comparação entre as diferentes abordagens, apresenta-se na Tabela 2.1 uma síntese comparativa das três principais ferramentas de ficção interativa baseadas nos critérios extraídos da literatura e da prática de desenvolvimento.

Tabela 2.1: *Análise comparativa de motores de ficção interativa.*

Critério	Twine	Ink/Inky	ChoiceScript
Licença	GPL v3 — livre (incl. comercial)	MIT — totalmente livre	CSL — não-comercial sem licença
Interface de Edição	Visual (grafo de nós/arestas)	Textual (scripting em IDE)	Textual (scripting linear)
Exportação Standalone	HTML autónomo (sem engine)	Requer engine externa (Unity/Unreal)	Web via plataforma CoG
Adoção Académica/Indie	Muito elevada	Média (foco profissional)	Média (foco romances interativos)
Integração com Motores	Limitada (plugins de terceiros)	Nativa (Unity, Unreal — oficial)	Nenhuma
Ferramentas de Teste	Limitadas	Básicas	Avançadas (quicktest, randomtest)
Formato Aberto	Sim (Twee3 — spec pública)	Sim (JSON compilado)	Não (proprietário CoG)
Curva de Aprendizagem	Baixa (visual, sem código)	Alta (linguagem scripting)	Média (sintaxe simples)
Mapeamento para Grafo	Direto (passagem = nó, link = aresta)	Indireto (estrutura textual)	Indireto (estrutura textual)

A seleção dos critérios de avaliação apresentados na Tabela 2.1 justifica-se pela necessidade de analisar de que forma as soluções existentes respondem aos desafios práticos da autoria e produção de FI:

- **Licenciamento (Licença):** Define a liberdade comercial e legal do autor. Licenças livres como a GPL v3 (Twine) ou MIT (Ink) promovem a autonomia e a publicação sem restrições, ao passo que licenças proprietárias como a CSL (ChoiceScript) impõem limites à distribuição comercial e exigências de royalties.
- **Interface de Edição e Mapeamento para Grafo:** Avaliam a carga cognitiva imposta ao autor durante a escrita. Um mapeamento direto em canvas visual (Twine) facilita o desenho de percursos geográficos, enquanto interfaces textuais (Ink e ChoiceScript) exigem que o escritor mantenha mentalmente o fluxo de caminhos ramificados.
- **Exportação Standalone (Distribuição) e Integração com Motores:** Definem a flexibilidade técnica da obra final. A exportação para HTML autónomo (Twine) facilita a partilha web imediata, enquanto a integração nativa com motores de jogo como Unity e Unreal (Ink) é indispensável para produções comerciais de grande escala.
- **Curva de Aprendizagem e Adoção Académica/Indie:** Refletem a acessibilidade para autores não-técnicos. Ferramentas sem código (Twine) apresentam baixa barreira de entrada e elevada popularidade na comunidade independente, enquanto linguagens de scripting avançadas (Ink) exigem conhecimentos de lógica de programação.
- **Ferramentas de Teste e Garantia de Qualidade:** Em narrativas complexas com centenas de caminhos, a validação manual é inviável. A existência de testes automáticos baseados em aleatoriedade (como o `randomtest` do ChoiceScript) é crucial para detetar incoerências lógicas e becos sem saída.
- **Formato Aberto:** Determina se o código e a estrutura do projeto pertencem a um padrão livre documentado (como o Twee 3 no Twine) que evite a dependência de plataformas proprietárias e garanta a perenidade do trabalho.

2.4 Integração de Modelos de Linguagem de Grande Escala (LLMs) Locais

Com o avanço recente no campo da Inteligência Artificial Generativa, a integração de Modelos de Linguagem de Grande Escala (*Large Language Models* - LLMs) tornou-se um recurso valioso para auxiliar autores na expansão automática e co-criação de narrativas interativas. No entanto, para garantir a viabilidade prática do projeto PiaP sem dependências financeiras ou de conectividade externa, focou-se a pesquisa em modelos de execução local e gratuita.

2.4.1 Análise de Famílias de Modelos e Recomendações

Foram analisadas três das principais famílias de modelos em open-weight ajustados para instruções, avaliando a sua capacidade e adequabilidade para autoria de FI:

1. Família Llama 3 e 3.1 (Meta AI):

- **Versão Recomendada:** Llama 3.1 (8B).
- **Justificação:** Representa um dos modelos leves mais capazes do estado da arte. A versão de 8 mil milhões de parâmetros corre com excelente desempenho na maioria dos computadores modernos equipados com placa gráfica dedicada ou em arquiteturas Mac Apple Silicon (M1/M2/M3), requerendo investimentos de hardware reduzidos.
- **Vantagem Narrativa:** Demonstra uma capacidade excecional no cumprimento estrito de diretrizes de formatação (*system prompts* rígidos). Esta fiabilidade torna-o ideal para gerar respostas estruturadas (por exemplo, em formato JSON contendo a sugestão de cena e as escolhas de saída), facilitando a integração direta no motor lógico do software.

2. Família Mistral (Mistral AI):

- **Versão Recomendada:** Mistral NeMo (12B) ou Mistral v0.3 (7B).
- **Justificação:** Os modelos desenvolvidos pela empresa europeia Mistral AI são amplamente reconhecidos pela comunidade de escrita criativa pelo seu tom mais orgânico e natural. Adicionalmente, tendem a apresentar menor nível de censura prévia, facilitando a escrita livre em géneros literários como ficção geral, mistério ou terror.
- **Vantagem Narrativa:** Apresentam janelas de contexto extremamente eficientes. A janela de contexto atua como a memória de curto prazo do modelo; numa narrativa ramificada, é fundamental que o LLM recorde o histórico de cenas e as decisões anteriores tomadas pelo utilizador para garantir a coerência semântica global do texto gerado.

3. Gemma 2 (Google):

- **Versão Recomendada:** Gemma 2 (9B).
- **Justificação:** Apesar do seu tamanho compacto de 9 mil milhões de parâmetros, destaca-se pelo seu desempenho elevado em raciocínio abstrato e análises textuais comparativas.
- **Vantagem Narrativa:** É altamente eficaz na realização de resumos e sínteses. Esta capacidade é ideal para funcionalidades auxiliares que leiam um determinado ramo completo da história e gerem um resumo conciso (ex: “descreve os principais acontecimentos deste arco narrativo”).

3

Conceção e Implementação

Este capítulo descreve a arquitetura geral do *Plot in a Pot*, detalhando as decisões de design de dados, o funcionamento dos algoritmos de processamento, as estratégias de visualização e acessibilidade adotadas, e o motor de simulação implementado.

3.1 Arquitetura Geral do Sistema

O sistema foi desenhado segundo uma arquitetura modular dividida em três camadas conceptuais independentes de tecnologia. Esta abordagem garante a separação de responsabilidades entre a recolha de comandos, o processamento de dados e a sua persistência, promovendo a manutenibilidade e a evolução isolada de cada módulo:

1. **Camada de Apresentação (Interface):** Responsável por gerir a interação direta com o utilizador. É constituída por:
 - **Canvas de Grafos:** O espaço visual interativo onde o autor cria, manipula e liga os nós narrativos.
 - **Matriz de Tradução:** Uma grelha estruturada que expõe os dicionários de localização e permite a tradução de chaves de texto em tempo real.
 - **Ecrã do Simulador:** A interface interativa de jogo onde o autor testa e experimenta a narrativa sob o ponto de vista do leitor.
2. **Camada de Processamento Lógico (Núcleo):** Contém a inteligência do sistema e os motores de transformação. É constituída por:
 - **Parser Twee 3:** Módulo encarregue de ler ficheiros de texto estruturados em formato Twee 3 e convertê-los no modelo interno da aplicação, bem como exportá-los de volta.
 - **Validador Estático:** Motor que executa travessias de grafo (**BFS/DFS**) para inspecionar inconsistências estruturais, como ciclos sem saída, nós inacessíveis ou hiperligações quebradas.

- **Motor de Simulação:** Sandbox responsável por inicializar scripts customizados, interpretar condicionais lógicas e controlar o estado de jogo na simulação.
3. **Camada de Dados e Persistência:** Representa o modelo de informação e o armazenamento. É constituída por:
- **Lista de Adjacência:** O modelo de dados em memória que representa a topologia do grafo, as saídas (sucessores) e entradas (predecessores) de cada cena.
 - **Dicionários JSON:** A base de dados estruturada que armazena os dicionários de tradução e localização no armazenamento local.

O fluxo de funcionamento da aplicação e a relação de comunicação entre estas camadas encontram-se representados no diagrama detalhado da Figura 3.1.

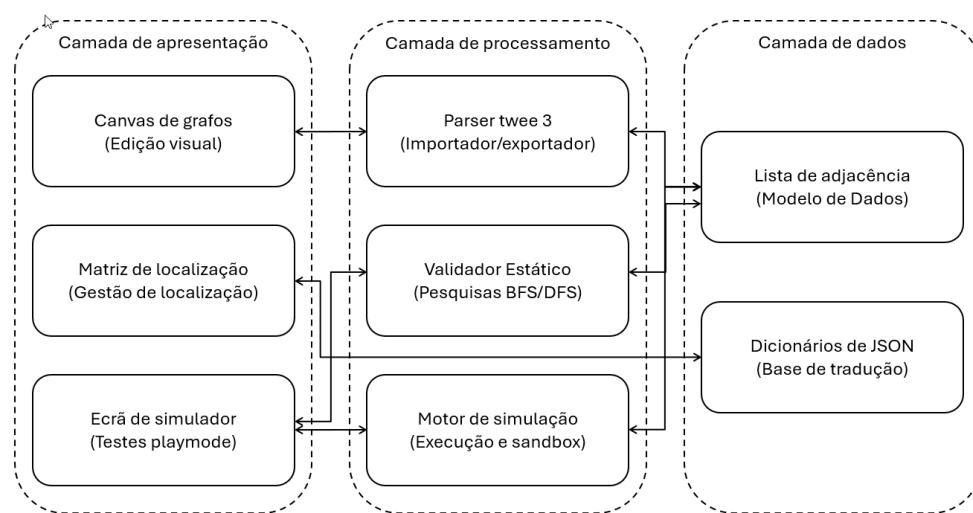


Figura 3.1: Diagrama de blocos detalhado da arquitetura conceptual e fluxo de dados.

3.2 Requisitos Funcionais e Não Funcionais

Para a materialização prática do sistema, os objetivos e requisitos gerais delineados na introdução foram detalhados sob a forma de requisitos técnicos. Estes dividem-se em funcionais (o que o sistema deve executar diretamente) e não funcionais (as restrições técnicas, de qualidade e usabilidade sob as quais o sistema deve operar).

3.2.1 Requisitos Funcionais

RF01 - Criação e Edição de Nós Narrativos (Cenas): O utilizador deve poder criar, mover, editar (alterando título e conteúdo de texto) e apagar nós no canvas interativo.

- RF02 - Ligações Direcionadas entre Nós (Escolhas):** O sistema deve ler ligações inseridas no texto através da sintaxe double-bracket (`[[TextoDestino]]`) e criar arestas direcionadas automaticamente no grafo.
- RF03 - Parser e Compatibilidade Twee 3:** O software deve importar e exportar ficheiros no formato textual normativo `.twee` síncrona e bidirecionalmente, preservando a sintaxe Twine e convertendo coordenadas.
- RF04 - Auditoria e Validação Estática de Fluxo:** O sistema deve executar análises estáticas sobre a estrutura do grafo em segundo plano para detetar e alertar sobre nós órfãos (inacessíveis), ligações quebradas (que apontam para nós apagados) ou ciclos infinitos sem condição de saída lógica.
- RF05 - Matriz de Localização Multi-idioma:** Disponibilizar uma tabela em formato de grelha (*grid*) que permita traduzir o texto das cenas para diferentes idiomas a partir de chaves dinâmicas, com suporte para exportação e importação em formato `CSV`.
- RF06 - Simulador em Tempo Real (PlayMode):** O utilizador deve poder testar a história jogando-a numa sandbox isolada dentro da aplicação que interprete as macros lógicas do SugarCube e execute código Javascript e CSS customizado.
- RF07 - Zonas de Agrupamento Visual:** O utilizador deve ser capaz de criar áreas retangulares redimensionáveis no canvas (Zonas) para agrupar nós de cena sob um mesmo contexto visual.
- RF08 - Importação Inteligente via Conversor LLM:** A aplicação deve permitir que o utilizador submeta uma história corrida em linguagem natural e, através de um modelo de linguagem local (Ollama) ou remoto (Gemini), a converta automaticamente em ficheiro no formato Twee 3 para gerar o respetivo grafo.

3.2.2 Requisitos Não Funcionais

- RNF01 - Portabilidade e Execução no Cliente (Browser):** A plataforma deve ser executável inteiramente no lado do cliente através de um navegador de internet padrão, sem requerer servidores ou infraestruturas complexas.
- RNF02 - Usabilidade e Acessibilidade Visual:** A interface de grafos deve utilizar a estética neo-brutalista e garantir que a visualização de fluxos (arestas de entrada e saída) seja distinguível sem dependência exclusiva de cor, apoiando utilizadores daltónicos.
- RNF03 - Desempenho e Fluidez:** O canvas do React Flow e a conversão do parser síncrono devem suportar dezenas de nós em simultâneo com latência de interação e atualização abaixo de 100 milissegundos.
- RNF04 - Privacidade e Soberania de Dados:** Todos os dados introduzidos, incluindo ficheiros importados, tabelas de tradução e dados submetidos para inteligência artificial no Ollama local, devem permanecer estritamente no armazenamento local do utilizador para garantir segurança absoluta.

3.3 Enquadramento e Implementação Tecnológica

Para concretizar as especificações da arquitetura conceptual descrita anteriormente, foi seleccionada uma pilha tecnológica assente em padrões modernos de desenvolvimento web do lado do cliente:

- **Framework Frontend (React 19):** A aplicação foi desenvolvida como uma Single Page Application (**Single Page Application (SPA)**) utilizando a biblioteca **React 19** (**Meta Open Source, 2013**). A escolha justifica-se pela elevada eficiência do mecanismo de atualização do DOM virtual e pela facilidade na gestão dinâmica de estados complexos (nós, arestas e variáveis). A reatividade nativa do React assegura a propagação imediata de edições textuais e lógicas sem sobrecarregar o processador do cliente.
- **Motor do Canvas de Grafos (ReactFlow 11):** O canvas interativo foi construído com a biblioteca **ReactFlow 11** (**webkid.io, 2021**). A sua seleção evitou o desenvolvimento de um motor de renderização de grafos a partir do zero, o qual seria complexo e propenso a erros de desempenho. O ReactFlow fornece suporte robusto para interações essenciais (arrastar, redimensionar, fazer zoom) e integra-se nativamente com componentes React customizados, viabilizando o desenho de nós de cena e Zonas personalizadas.
- **Estilização e Identidade Visual (Tailwind CSS 3):** A estética neo-brutalista foi implementada com **Tailwind CSS 3** (**Tailwind Labs, 2017**). A utilização de classes utilitárias acelerou significativamente a estilização de contornos espessos, sombras sólidas e regras de alto contraste necessárias para a acessibilidade visual da plataforma, garantindo uma folha de estilos limpa e de fácil manutenção sem necessidade de ficheiros CSS ad-hoc.
- **Motor de Internacionalização (i18next):** A biblioteca **i18next** foi seleccionada para a gestão de matrizes de localização, por constituir o padrão de facto para internacionalização em ecossistemas React. Permite o carregamento assíncrono de chaves **JSON** de tradução dinamicamente na memória local via requisições HTTP locais no navegador (através de **i18next-http-backend**), assegurando a separação limpa de idiomas e a segurança dos dados.

3.3.1 Justificação das Decisões de Design

Com base na análise comparativa das ferramentas existentes (ver Secção 2.3.7), as escolhas de design e de tecnologia para o **PiaP** foram fundamentadas nos seguintes pontos:

- **Fusão de Paradigmas (Visual vs. Textual):** Enquanto o Twine se destaca pela interface puramente visual e o Ink pelo controlo fino de estados lógicos complexos, o **PiaP** visa fundir ambas as forças, proporcionando um editor visual de grafos que não abdica da manipulação robusta de condicionais e variáveis (via macro-linguagem SugarCube).

- **Adoção do Formato Aberto Twee 3 e Limitação de Grafos:** A especificação aberta Twee 3 possui um mapeamento direto entre passagens e grafos, o que suporta a decisão de adotá-lo como base de persistência do **PiaP**. Contudo, para permitir que o sistema leia as saídas e as renderize visualmente como arestas de forma fiável, o projeto introduz um compromisso de design (*trade-off*) em relação ao Twine tradicional: não é suportada a utilização de variáveis como destinos dinâmicos de hiperligações (ex: `[[Ir para o local|$local]]`), exigindo que todos os destinos do grafo sejam estáticos.
- **Validação Automática e Segurança do Fluxo:** O ChoiceScript é valorizado pelas suas ferramentas robustas de teste automático (**randomtest**), uma lacuna crítica do Twine. O **PiaP** incorpora de raiz este princípio ao disponibilizar o validador estático (**BFS**) e o Smart Walker no simulador para preencher esta falta, garantindo a correção imediata do fluxo narrativo.
- **Autonomia e Licenciamento Livre:** A restrição de uso comercial imposta pelo ChoiceScript salienta a importância de desenvolver uma ferramenta que promova a autonomia dos autores, permitindo a exportação de projetos sob licenças livres e sem taxas adicionais.

3.4 Modelação de Dados: Lista de Adjacência Bidirecional

Para representar o fluxo da história no editor, evitou-se o uso de uma Matriz de Adjacência convencional devido ao desperdício de memória ($O(V^2)$) e à fraca escalabilidade com o crescimento do número de nós. Em vez disso, implementou-se uma **Lista de Adjacência Bidirecional** com complexidade $O(V + E)$.

Cada cena no modelo de dados armazena de forma independente dois conjuntos de caminhos:

1. **Saídas (Sucessores):** Um vetor que contém as escolhas explícitas disponíveis para o leitor, mapeando o texto da opção e o ID do nó de destino.
2. **Entradas (Predecessores):** Um vetor que regista as referências de todos os outros nós que contém escolhas a apontar para o nó atual.

Esta representação bidirecional permite à aplicação realizar mapeamento reverso instantâneo (*backward mapping*), fornecendo na barra lateral de propriedades os painéis “Vens de...” e “Vais para...”, facilitando ao autor a navegação e a validação rápida de interconexões sem ter de vasculhar visualmente o canvas.

A estrutura de dados **JSON** representativa de um nó e das suas ligações apresenta o seguinte formato:

```

1 {
2   "cenas": [
3     {
4       "id": "cena_praca",
5       "titulo": "A Praça Central",

```

```

6     "conteudo": "O sol brilha no mercado central.",
7     "saidas": [
8         { "texto_escolha": "Entrar na taberna", "destino": "cena_taberna" },
9         { "texto_escolha": "Aguardar um momento", "destino": "cena_praca" }
10    ],
11    "entradas": ["cena_praca", "cena_taberna"]
12 },
13 {
14     "id": "cena_taberna",
15     "titulo": "Interior da Taberna",
16     "conteudo": "O cheiro a vinho enche o ar.",
17     "saidas": [
18         { "texto_escolha": "Voltar lá fora", "destino": "cena_praca" }
19     ],
20     "entradas": ["cena_praca"]
21 }
22 ]
23 }

```

3.5 Parser Síncrono e Especificação Twee 3

O analisador desenvolvido em `tweeParser.js` (encontrado em `/src/utils/tweeParser.js`) realiza o mapeamento síncrono em tempo real entre a interface gráfica do React Flow e a representação textual no formato Twee 3 ([Interactive Fiction Technology Foundation, 2020](#)).

3.5.1 Importação e Conversão de Coordenadas em Zonas

No formato Twee 3 ([Interactive Fiction Technology Foundation, 2020](#)), as coordenadas de posição guardadas nos metadados **JSON** das passagens são sempre absolutas no espaço do ecrã:

$$(X_{\text{absoluto}}, Y_{\text{absoluto}})$$

Quando um nó é importado como filho de uma Zona de Agrupamento Visual (*ZoneNode*), o React Flow exige que a sua posição seja definida em coordenadas relativas ao canto superior esquerdo da zona mãe. O parser realiza esta conversão no momento da leitura:

$$X_{\text{relativo}} = X_{\text{absoluto}} - X_{\text{zona}}$$

$$Y_{\text{relativo}} = Y_{\text{absoluto}} - Y_{\text{zona}}$$

Ao exportar o grafo novamente para o formato `.twee`, o parser faz a conversão inversa para garantir a compatibilidade com outros compiladores de Twine tradicionais:

$$X_{\text{absoluto}} = X_{\text{zona}} + X_{\text{relativo}}$$

$$Y_{\text{absoluto}} = Y_{\text{zona}} + Y_{\text{relativo}}$$

3.5.2 Passagens Especiais

O parser reconhece passagens reservadas de metadados da especificação Twee 3:

- `:: StoryTitle`: Define o título do projeto.
- `:: StoryData`: Contém as configurações globais em formato **JSON**, incluindo o identificador único da história (`ifid`), o formato de compilação (**SugarCube**), o nó de arranque (`start`) e o nível de zoom.
- `:: StoryInit`: Nó especial não-narrativo onde são declaradas e inicializadas as variáveis de controlo de estado da simulação.

3.5.3 Modelo SugarCube (Ligações e Macros)

As arestas são extraídas do texto narrativo de cada nó através de expressões regulares que identificam ligações no formato clássico:

- **Arestas Simples**: `[[Cena2]]`
- **Arestas com Alias/Texto de Opção**: `[[Ir para a Taverna|cena_taberna]]` ou `[[Abrir porta->cena_interior]]`

As alterações de estado são definidas em macros como `<<set $moedas += 10>>`, e as condições lógicas que limitam a visibilidade de opções são declaradas através de blocos `<<if $tem_chave>> ... <</if>>`.

3.6 Interação Visual, Edição e Acessibilidade

3.6.1 Zonas de Agrupamento Visual

A implementação de Zonas como nós customizados de grupo (*ZoneNode*) exigiu uma resolução específica para um problema clássico de captura de cliques no React Flow. Como o contentor nativo das Zonas cobre uma área retangular considerável sobrepondo-se aos nós de cena interiores, os pointer events capturavam os cliques na tela, impedindo o arrasto ou seleção dos nós filhos.

A solução consistiu em definir a propriedade CSS `pointer-events: none` dinamicamente na configuração global do React Flow para o contentor da Zona em `App.jsx` (encontrado em `/src/App.jsx`). Em paralelo, no componente `ZoneNode.jsx` (encontrado em `/src/components/ZoneNode.jsx`), forçou-se `pointer-events: auto` exclusivamente nas pegadas de redimensionamento (*NodeResizer*) e na barra de cabeçalho da zona (utilizada para arrastar a zona em bloco com todos os seus filhos).

Para além do agrupamento espacial, as Zonas e os nós customizados suportam a injeção de imagens de fundo personalizadas (*backgrounds*). Esta funcionalidade é gerida reativamente através de uma propriedade `bgImage` armazenada nos metadados de estado de cada nó. O renderizador do React Flow decodifica este campo em tempo de

execução e aplica a imagem como máscara de fundo utilizando estilos embutidos do CSS, permitindo categorizar regiões do canvas visualmente.

3.6.2 Arestas Curvadas com Waypoints

Para evitar a sobreposição caótica de ligações direcionadas e o cruzamento cego sobre os nós de cena (que cria um efeito visual denso e desorganizado), a plataforma implementa arestas customizadas que utilizam a interpolação por Splines Cúbicas de Catmull-Rom (Catmull et al., 1974) em `EditableEdge.jsx` (encontrado em `/src/components/EditableEdge.jsx`).

O autor pode introduzir e arrastar marcadores circulares (*waypoints*) interativos ao longo da linha no canvas do React Flow. A curva é calculada em tempo de execução interpolando sequencialmente estes waypoints de modo a gerar uma trajetória suave e contínua.

Formalmente, dada uma sequência de pontos de controlo tridimensionais ou bidimensionais no plano do canvas $\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_n$, a spline cúbica de Catmull-Rom para um segmento específico i que conecta os pontos $\mathbf{P}_i$ e $\mathbf{P}_{i+1}$ (utilizando $\mathbf{P}_{i-1}$ e $\mathbf{P}_{i+2}$ como os pontos de controlo auxiliares para definir as tangentes nos extremos) é parametrizada por uma variável $t \in [0, 1]$. O vetor de posição da curva $\mathbf{C}(t)$ é dado por:

$$\mathbf{C}(t) = \frac{1}{2} \begin{bmatrix} 1 & t & t^2 & t^3 \end{bmatrix} \mathbf{M}_{CR} \begin{bmatrix} \mathbf{P}_{i-1} \\ \mathbf{P}_i \\ \mathbf{P}_{i+1} \\ \mathbf{P}_{i+2} \end{bmatrix}$$

onde $\mathbf{M}_{CR}$ representa a matriz de coeficientes característica da Spline de Catmull-Rom centripeta padrão (com $\alpha = 0.5$ para obter curvas sem auto-intersecções ou loops):

$$\mathbf{M}_{CR} = \begin{bmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{bmatrix}$$

Desta forma, expandindo a multiplicação matricial, cada coordenada (x, y) da spline é calculada de forma explícita através de equações polinomiais cúbicas em t :

$$\mathbf{C}(t) = \frac{1}{2} \left[2\mathbf{P}_i + (-\mathbf{P}_{i-1} + \mathbf{P}_{i+1})t + (2\mathbf{P}_{i-1} - 5\mathbf{P}_i + 4\mathbf{P}_{i+1} - \mathbf{P}_{i+2})t^2 + (-\mathbf{P}_{i-1} + 3\mathbf{P}_i - 3\mathbf{P}_{i+1} + \mathbf{P}_{i+2})t^3 \right]$$

Este cálculo polinomial é executado em JavaScript na biblioteca gráfica `graphMath.js` para interpolar os waypoints introduzidos pelo utilizador. Visto que os navegadores web não conseguem renderizar splines de Catmull-Rom de forma nativa na especificação SVG, o módulo de visualização converte os segmentos de reta calculados em comandos de curvas Bézier cúbicas padrão (sintaxe do caminho SVG com as flags **C** e **S**). A conversão é feita calculando analiticamente os pontos de controlo Bézier correspondentes com base

nas tangentes geradas pela formulação de Catmull-Rom em cada nó:

$$\mathbf{B}_{control1} = \mathbf{P}_i + \frac{\mathbf{P}_{i+1} - \mathbf{P}_{i-1}}{6}$$

$$\mathbf{B}_{control2} = \mathbf{P}_{i+1} - \frac{\mathbf{P}_{i+2} - \mathbf{P}_i}{6}$$

Isto garante a renderização direta no navegador web por aceleração de hardware (GPU), mantendo a fluidez de arrasto e a complexidade temporal linear $O(W)$ (onde W é o número de waypoints da aresta).

3.6.3 Gestão de Variáveis Estilo Figma

O componente `VariablesModal.jsx` (encontrado em `/src/components/VariablesModal.jsx`) centraliza a gestão de variáveis globais e locais da história através de uma interface inspirada na ferramenta Figma. Quando uma variável é renomeada no painel global (ex: de `$ouro` para `$moedas`), um mecanismo automático executa uma expressão regular compilada:

```
1 new RegExp('\\$' + antigo + '\\b', 'g')
```

Esta regex varre e atualiza em tempo real todas as referências da variável dentro das macros `<<set>>`, `<<if>>` e inserções de texto em todos os nós da aplicação, prevenindo a quebra de referências em projetos grandes.

3.6.4 Acessibilidade para Autores com Dificuldades Visuais

Tendo em conta que um dos autores da ferramenta, Miguel, apresenta limitações visuais (nomeadamente daltonismo), desenhou-se uma estratégia de representação de fluxo e interface adaptada. A escolha da estética neo-brutalista para a interface fundamenta-se no facto de este estilo, quando implementado com rigor técnico, favorecer a acessibilidade de utilizadores com deficiências visuais (Babich, 2023; Loranger, 2022), através de:

- **Contraste Elevado e Bordas Espessas:** O neo-brutalismo recorre a contornos pretos e espessos (de 2px ou mais) para delimitar elementos de interface (como botões e caixas de texto) e sombras projetadas sólidas e sem difusão, o que facilita a distinção visual de contêineres e controlos interativos por parte de pessoas com baixa acuidade visual.
- **Tipografia Legível:** Uso de fontes sem serifa com pesos elevados (*bold* e *black*) que evitam o esbatimento visual do texto.
- **Arestas de Alto Contraste:** As ligações entre nós utilizam cores de alto contraste (pretas no tema claro, brancas no tema escuro) e são sempre sólidas, garantindo excelente visibilidade sobre o fundo do canvas independentemente do tema ativo.

- **Sinalização Semântica Redundante:** Cores de contraste elevado e símbolos iconográficos explícitos acompanham o feedback visual em detrimento de códigos de cores simples (verde/vermelho).

3.6.5 Modelação de Dados e Sincronização com o React Flow

Para além da representação puramente abstrata da arquitetura modular, a implementação prática exige uma sincronização bidirecional em tempo real entre a interface gráfica gerida pela biblioteca *React Flow* e a estrutura de dados semântica interna da história.

No **PiaP**, o estado centralizado da narrativa é mantido na raiz da aplicação (**App.jsx**) e modularizado através de ganchos customizados (*custom hooks*), recorrendo a três vetores de dados estruturados em memória local:

1. **Vetor de Nós do React Flow (nodes):** Contém a representação posicional e de renderização de cada elemento no canvas. Cada objeto de nó segue a especificação rígida da biblioteca:

```

1    {
2      id: "cena_taberna",
3      type: "storyNode",
4      position: { x: 250, y: 180 },
5      data: {
6        title: "Interior da Taberna",
7        content: "O cheiro a hidromel preenche a sala...",
8        tags: ["secreto"],
9        bgImage: ""
10     },
11     width: 280,
12     height: 150
13   }
```

Se o nó representar uma Zona de Agrupamento (**ZoneNode**), o tipo é alterado para `'zoneNode'` e o objeto passa a incluir propriedades de agrupamento como `parentId` nos nós filhos e dimensões explícitas de largura e altura controladas pelo utilizador.

2. **Vetor de Arestas do React Flow (edges):** Modela a conectividade geométrica das escolhas narrativas. Cada aresta é configurada como um componente customizado (**EditableEdge**) e armazena os pontos geométricos intermédios (*waypoints*) inseridos pelo autor para suavizar as curvas:

```

1    {
2      id: "edge-cena_praca-cena_taberna",
3      source: "cena_praca",
4      target: "cena_taberna",
5      type: "editableEdge",
6      data: {
```

```

7      choiceText: "Entrar na taberna",
8      waypoints: [{ x: 120, y: 150 }, { x: 180, y: 160 }]
9    }
10  }

```

3. **Lista de Adjacência Bidirecional (`adjacencyList`):** Representa o grafo direcionado de estados do ponto de vista do motor lógico. É recalculada de forma síncrona a cada modificação nos vetores de nós ou arestas. Esta lista mapeia cada identificador de cena ao seu conjunto de sucessores (saídas) e predecessores (entradas), permitindo ao validador e ao motor de simulação executar travessias e lookups rápidos de transição com complexidade constante $O(1)$.

Esta tripla modelagem é sincronizada de forma atômica. Quando o autor arrasta um nó ou altera o texto de uma escolha no inspetor, os manipuladores de eventos reativos do React Flow (como `onNodesChange` ou `onEdgesChange`) capturam a alteração física. Esta alteração é processada e encaminhada pelos hooks customizados em `App.jsx`, gerando novos vetores imutáveis de estado que forçam a reatualização dos componentes visuais e o recálculo imediato da lista de adjacências e do mapa de erros lógicos.

3.7 Validação Estática e Simulação

O motor de autoria do *Plot in a Pot* integra ferramentas robustas de validação e execução reativa. Para além de permitir a escrita de histórias, a plataforma disponibiliza um sistema de verificação lógica que audita a estrutura narrativa de forma estática e uma sandbox de simulação dinâmica dotada de agentes de navegação autónoma.

3.7.1 Algoritmo de Validação Estática e Exploração do Espaço de Estados (BFS)

O validador estático, implementado em `storyValidator.js` e `storyTraversal.js`, resolve o problema de acessibilidade e integridade da narrativa. Em grafos estáticos simples, uma travessia clássica de Pesquisa em Largura (BFS) seria suficiente para auditar o grafo. Contudo, em FI, as arestas (hiperligações) dependem frequentemente de expressões lógicas e variáveis de estado (macros `<<if>>` e `<<set>>` do SugarCube). A acessibilidade de um nó v não depende apenas do caminho geométrico, mas também da valoração atual das variáveis de estado (memória).

Assim, a validação lógica é modelada como um problema de *Exploração do Espaço de Estados* (*State Space Exploration*), onde o grafo de transições real é constituído por pares $\langle v, \sigma \rangle$, nos quais $v \in \mathcal{N}$ representa o nó do editor e $\sigma : \mathcal{V} \rightarrow \mathcal{D}$ representa um mapeamento de variáveis lógicas globais para os seus respetivos domínios de valores.

O algoritmo executa a travessia de acordo com os seguintes passos estruturados:

1. **Inicialização e Pré-computação:** Para garantir a eficiência temporal ($O(1)$ em lookups), o algoritmo pré-computa em memória:

- Um mapa de nós $NodeMap : ID \rightarrow Node$, indexando todos os vértices da narrativa.
- Um mapa de adjacências de arestas $EdgesBySource : ID \rightarrow List\langle Edge \rangle$.

O ponto de entrada v_0 é resolvido por $findStartNode(NodeMap)$, e o estado inicial σ_0 é lido a partir das declarações de variáveis do nó de sistema $StoryInit$.

2. **Estrutura de Dados e Fila:** Uma fila síncrona Q é instanciada e inicializada com o tuplo de arranque:

$$Q \leftarrow \text{enqueue}(Q, \langle v_0, \sigma_0, \text{path} = [v_0.\text{label}] \rangle)$$

São criados conjuntos vazios para registar nós alcançáveis ($\mathcal{N}_{reach}$), arestas alcançáveis ($\mathcal{E}_{reach}$), e tabelas hash de estados visitados por nó ($VisitedStates : ID \rightarrow Set\langle String \rangle$) para evitar caminhos redundantes e ciclos infinitos.

3. **Ciclo de Processamento Principal:** Enquanto Q não estiver vazia, remove-se o primeiro elemento:

$$\langle u, \sigma, \text{path} \rangle \leftarrow \text{dequeue}(Q)$$

Para o nó corrente u , efetua-se o seguinte processamento:

- (a) Adiciona-se o identificador de u a $\mathcal{N}_{reach}$.
 - (b) Executam-se as instruções de modificação de estado contidas no texto do nó (por exemplo, macros de atribuição $\langle\langle \text{set} \rangle\rangle$), produzindo um novo estado de memória $\sigma' \leftarrow \text{applyModifiers}(u.\text{content}, \sigma)$.
 - (c) Verifica-se se o estado σ' já foi visitado no nó u . Para tal, converte-se σ' numa representação string serializada $S_{\sigma'}$ e verifica-se se $S_{\sigma'} \in VisitedStates[u]$. Em caso afirmativo, o ramo é podado (*pruning*) para evitar ciclos redundantes no grafo.
 - (d) Para evitar a explosão do espaço de estados em loops que incrementam variáveis infinitamente, impõe-se um limite de segurança de $K = 10$ estados únicos por nó. Se $|VisitedStates[u]| \geq K$, o algoritmo emite um aviso de ciclo infinito no terminal e poda o caminho.
 - (e) Caso contrário, insere-se $S_{\sigma'}$ em $VisitedStates[u]$ e regista-se a rota percorrida e o estado final na tabela de histórico de chegada $ArrivalHistory[u]$.
4. **Avaliação de Transições e Condicionais:** Para cada aresta de saída $e \in EdgesBySource[u]$ que conecta o nó u ao nó de destino w :
 - (a) O validador extrai a hiperligação textual associada à aresta e do conteúdo de u .
 - (b) Avalia-se a condição de acessibilidade associada a essa escolha através da função $canAccessChoice(u.\text{content}, \text{link}, \sigma')$. Esta função analisa se o link está envolto por alguma macro condicional $\langle\langle \text{if} \rangle\rangle$ e avalia a sua expressão lógica contra a memória σ' .
 - (c) Se a condição for satisfeita (*true*), adiciona-se o ID da aresta e a $\mathcal{E}_{reach}$ e

enfileira-se o próximo estado:

$$Q \leftarrow \text{enqueue}(Q, \langle w, \sigma', \text{path} \cup [w.\text{label}] \rangle)$$

Após o término da travessia (quando a fila Q fica vazia), o módulo de auditoria em `storyValidator.js` pós-processa os resultados recolhidos para isolar as falhas estruturais, categorizando-as em:

- **Nós Órfãos** (V_{orphan}): Nós $v \in \mathcal{N}$ que não pertencem a $\mathcal{N}_{\text{reach}}$ e cujas etiquetas não incluem a tag `'secreto'`.
- **Becos sem Saída** ($V_{\text{dead_end}}$): Nós $v \in \mathcal{N}_{\text{reach}}$ que contêm hiperligações cujo destino aponta para um ID de nó inexistente no grafo ($w \notin \mathcal{N}$).
- **Arestas Inacessíveis** ($E_{\text{unreachable}}$): Arestas $e \in \mathcal{E}$ que não pertencem a $\mathcal{E}_{\text{reach}}$. Estas representam caminhos cujas condicionais lógicas nunca são satisfeitas sob nenhuma das valorações de variáveis exploradas.

3.7.2 Simulador em Tempo Real (PlayMode) e Smart Walker

O simulador de jogo (`PlayMode.jsx`) mimetiza o comportamento do interpretador SugarCube num ambiente controlado pelo navegador do autor. Para permitir a depuração em tempo real sem afetar os dados de edição, o motor executa sob um isolamento de contextos:

- **Separação de Estilos:** Os nós categorizados com a tag `css` têm o seu conteúdo concatenado em tempo de execução e injetado sob a forma de uma tag `<style id='playmode-styles'>` na raiz do cabeçalho da página, sendo totalmente limpos ao sair da simulação.
- **Isolamento de Scripts:** Os nós lógicos contendo JavaScript são processados como funções anónimas com escopo isolado através do construtor `new Function('state', 'State', 'setup', script_content)`, impedindo que variáveis globais escritas pelo utilizador interfiram com os estados internos do React ou do canvas do React Flow.

Para testes automáticos de caminhos narrativos, a plataforma disponibiliza o agente autónomo **Smart Walker** (implementado em `storyTraversal.js`). Em vez de seleccionar opções de forma puramente pseudo-aleatória (o que levaria a uma baixa cobertura de caminhos e retenção em ciclos curtos), o agente utiliza uma heurística baseada em *Procura por Novidade* (*Novelty Search*).

Dada a vizinhança do nó atual v , correspondente ao conjunto de nós adjacentes alcançáveis $N(v)$, o agente consulta o histórico de visitas acumuladas do simulador. Para cada opção de transição que leva ao nó vizinho $w \in N(v)$, o Smart Walker obtém o número acumulado de visitas históricas $H(w)$ registado na memória global do simulador. O Walker selecciona a transição para o nó vizinho w_{next} que minimiza a frequência de

visitas:

$$w_{next} = \arg \min_{w \in N(v)} H(w)$$

Este comportamento força o simulador a procurar caminhos narrativos raros e ramos profundos da história de forma automatizada, conseguindo mapear e encontrar becos sem saída ou ciclos lógicos fechados numa fração de segundo sem intervenção manual do autor.

3.7.3 Motor de Compilação JIT de Macros SugarCube

Para suportar as avaliações e atribuições de variáveis de estado descritas, a aplicação possui um compilador *Just-In-Time* (JIT) simplificado estruturado em `logicParser.js` e `sugarcubeLogic.js`:

- **Análise Léxica e Mapeamento:** O interpretador analisa o conteúdo textual dos nós através de varrimentos que procuram padrões de macros `<<set>>` e `<<if>>`. Traduz os operadores clássicos da linguagem SugarCube para os operadores lógicos válidos de JavaScript, conforme resumido na Tabela 3.1.
- **Árvore de Sintaxe Abstrata (Árvore de Sintaxe Abstrata (Abstract Syntax Tree) (AST)):** O interpretador de condicionais analisa blocos lógicos aninhados construindo uma AST simples que reflete a precedência de cláusulas. Esta estrutura é então avaliada em tempo de execução para calcular o valor de verdade das expressões.

Tabela 3.1: Mapeamento de operadores lógicos SugarCube para JavaScript.

Operador SugarCube	Operador JavaScript	Significado
is / eq	===	Igualdade estrita
neq	!==	Diferença estrita
and	&&	E lógico
or		OU lógico
to	=	Atribuição de valor
\$variavel	state.variavel	Acesso ao estado reativo

3.8 Integração de Modelos de Linguagem (LLMs)

Para além do núcleo lógico de grafos e da simulação estática, a plataforma disponibiliza um assistente baseado em Modelos de Linguagem de Grande Escala (LLMs) para facilitar a importação e co-criação de histórias. A conceção e implementação desta funcionalidade assenta nos seguintes aspetos:

3.8.1 Modelo de Integração Local sem Custos

Para evitar a necessidade de desenvolver e compilar motores de inferência de raiz, o **PiaP** adota uma arquitetura baseada em servidores de inferência locais prontos a usar, nomeadamente o *Ollama* ou o *LM Studio*. O fluxo de comunicação processa-se conforme descrito de seguida:

1. **Instalação Local:** O autor/utilizador instala o servidor *Ollama* na sua máquina local.
2. **Execução em Segundo Plano:** O *Ollama* descarrega o modelo pretendido (ex: `llama3.1:8b`) e executa a inferência local em segundo plano.
3. **Simulação de **Application Programming Interface (API)**:** O servidor expõe uma **API** REST local na porta padrão (tipicamente `http://localhost:11434`), que simula a interface padrão de chat da OpenAI.
4. **Pedidos HTTP:** O software *Plot in a Pot* efetua pedidos HTTP padrão (*fetch*) diretamente para este endpoint local, eliminando custos de infraestrutura e limites de chamadas impostos por **APIs** proprietárias na nuvem.

A escolha de investir no desenvolvimento de soluções de inferência locais (via *Ollama*) ao invés de recorrer a **APIs** de grandes fornecedores na nuvem (como OpenAI ou Anthropic) baseou-se em duas premissas estratégicas essenciais:

- **Privacidade e Soberania dos Dados:** Ao executar o modelo localmente no computador do autor, garante-se que toda a propriedade intelectual da história e os textos criados permanecem estritamente confidenciais. Não há partilha de dados com servidores externos de terceiros nem risco de os textos literários serem recolhidos para treino de modelos comerciais.
- **Custo Financeiro Zero:** **APIs** na nuvem implicam faturação contínua baseada em consumo de tokens (tanto para entrada como para saída), o que criaria barreiras ao uso regular e exigiria a gestão de chaves de **API** pagas por parte do utilizador. A inferência local permite a utilização gratuita e ilimitada da funcionalidade de conversão inteligente.

3.8.2 Papel do LLM: Conversão e Importação Automática de Histórias

Uma conclusão crítica retirada da pesquisa reside no facto de que os LLMs não possuem fiabilidade suficiente para gerir a coerência estrutural de um grafo de estado de forma autónoma e dinâmica em tempo de execução. Os modelos de linguagem tendem a alucinar, esquecer nós, criar ligações logicamente impossíveis, ignorar restrições de ciclos e perder o mapeamento geral da árvore narrativa quando integrados no fluxo direto de jogo.

Deste modo, no produto final, a função do LLM foi redefinida para atuar exclusivamente como uma ferramenta de conversão estrutural numa fase de pré-processamento. Em vez de ser integrado diretamente na simulação, o modelo é utilizado para ler uma

história corrida escrita pelo utilizador em linguagem natural e convertê-la inteiramente para o formato normativo Tweep 3 (identificando as passagens/nós, os seus títulos e as arestas de ligação). Uma vez gerada a representação textual Tweep, a aplicação importa-a de forma automática para o programa. A partir desse ponto, o motor lógico proprietário e o validador estático (baseados em listas de adjacências) assumem a responsabilidade exclusiva de desenhar o canvas visual e garantir a coerência matemática do grafo de estados.

4

Desenvolvimento e Avaliação

Este capítulo detalha o processo de desenvolvimento prático do *Plot in a Pot*, descrevendo as fases de implementação, o cronograma cronológico do projeto e a metodologia e resultados dos testes de utilizador realizados para validar a usabilidade e acessibilidade do sistema.

4.1 Cronograma e Fases de Desenvolvimento

O projeto foi executado ao longo de um período aproximado de três meses, utilizando uma abordagem de desenvolvimento incremental assente em prototipagem rápida. O cronograma de implementação dividiu-se em cinco fases fundamentais, estruturadas conforme se descreve de seguida:

Tabela 4.1: *Cronograma detalhado das fases de desenvolvimento do projeto.*

Fase	Descrição das Atividades Realizadas	Intervalo de Datas
Fase 1	Levantamento de requisitos e desenho conceitual da arquitetura.	17/03/2026 – 31/03/2026
Fase 2	Programação do núcleo (<i>core</i>) lógico: parser Twee 3 e validador BFS .	01/04/2026 – 25/04/2026
Fase 3	Interface e visualização: integração do React Flow e <i>waypoints</i> nas arestas.	26/04/2026 – 20/05/2026
Fase 4	Integração de IA local (Ollama) e otimização de acessibilidade visual.	21/05/2026 – 10/06/2026
Fase 5	Testes com utilizadores, correção de erros e refinamento final.	11/06/2026 – 25/06/2026

Fase 1 – Desenho e Requisitos: Definição das especificações funcionais e não funcionais do sistema. Elaboração do diagrama de arquitetura modular para separar a camada gráfica do processamento de grafos.

Fase 2 – Motor Lógico: Implementação da biblioteca `tweeParser.js` para ler e estruturar o formato Twee 3 em memória como uma lista de adjacências bidirecional,


```

| +- useChoiceSync.js          # Lógica de sincronização de escolhas
| +- useGraphHandlers.js       # Tratamento de eventos do React Flow
| +- useKeyboardShortcuts.js   # Atalhos globais de teclado
| +- useNodeOperations.js      # Operações CRUD em nós
| +- useProjectPersistence.js  # Auto-save e importação/exportação
| +- useSettings.js            # Estado de definições persistidas
| \- useUndoRedo.js            # Pilhas de Undo/Redo atômicas
+- utils/
| +- aiGenerator.js            # API REST para Ollama/Gemini
| +- graphMath.js              # Controlo geométrico e Splines
| +- logicParser.js            # Parser AST de macros SugarCube
| +- storySimulator.js         # Máquina de estados da história
| +- storyTraversal.js         # Exploração heurística (Smart Walker)
| +- storyValidator.js         # Validação estática com BFS
| +- sugarcubeLogic.js         # Compilador JIT de macros Twine
| +- tweeParser.js            # Parser bidirecional Tweep 3
| \- textParsing.js           # Funções utilitárias de parsing de texto
+- App.jsx                     # Orquestrador reativo principal
\- index.js                    # Entrada da aplicação

```

4.2.1 Componentes de Interface (src/components/)

Nesta secção, detalham-se os aspetos de desenvolvimento e engenharia aplicados à implementação dos principais componentes visuais da aplicação.

App.jsx: O Controlador Reativo Central e Sincronização de Estado

O ficheiro `App.jsx` atua como o orquestrador principal da aplicação. Inicialmente concebido como um controlador centralizado contendo toda a lógica de estado, foi refatorado para uma arquitetura modular composta por ganchos customizados (*custom hooks*) e funções utilitárias puras. Sob esta nova estrutura, o `App.jsx` coordena a montagem do grafo delegando as responsabilidades específicas de gestão de definições (`useSettings`), controlo de histórico (`useUndoRedo`), sincronização de escolhas (`useChoiceSync`), operações CRUD de nós (`useNodeOperations`), persistência do projeto (`useProjectPersistence`) e atalhos de teclado (`useKeyboardShortcuts`). Para além de implementar as rotinas de ligação com o React Flow, a gestão da pilha de histórico (`undoStack` e `redoStack`) foi isolada em `useUndoRedo.js` para permitir o retrocesso ou avanço atómico de edições na interface gráfica. Dado que o React Flow atualiza objetos de nós e arestas por referência, a criação de instantâneos (*snapshots*) do estado requer uma clonagem profunda (*deep clone*) para evitar que estados futuros partilhem a mesma referência de memória com o histórico, como demonstrado na Listagem 1.

Esta modularização dividiu as responsabilidades do controlador em ganchos específicos, promovendo a separação de conceitos (*Separation of Concerns*) e facilitando a

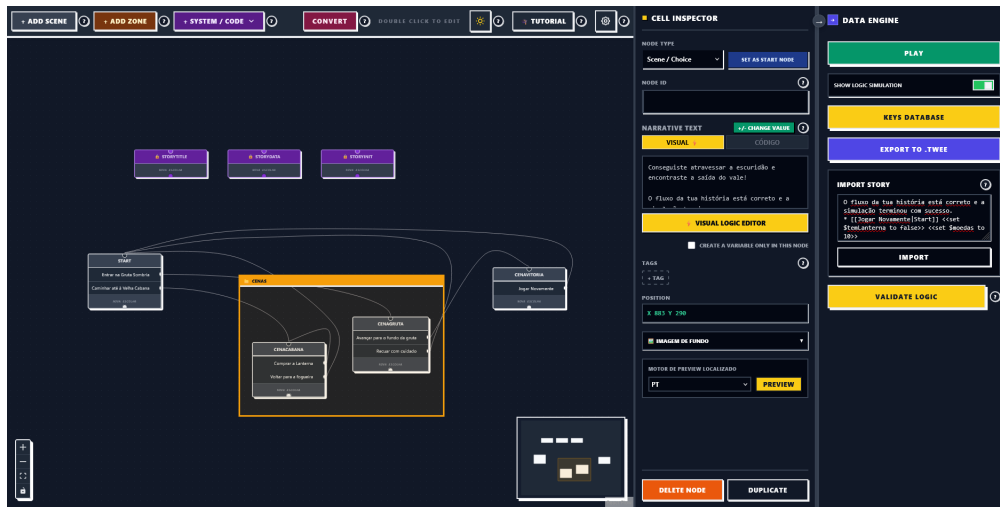


Figura 4.1: Interface gráfica do canvas de grafos do Plot in a Pot, demonstrando a estética neo-brutalista de alto contraste com nós de cena e agrupamentos visuais (Zonas).

```

1 // Registo de snapshots e funcoes de controlo em useUndoRedo.js
2 const takeSnapshot = useCallback(() => {
3   setPast(prev => [
4     ...prev.slice(-50),
5     {
6       nodes: JSON.parse(JSON.stringify(nodesRef.current)),
7       edges: JSON.parse(JSON.stringify(edgesRef.current))
8     }
9   ]);
10  setFuture([]);
11 }, []);
12
13 const undo = useCallback(() => {
14   if (past.length === 0) return;
15   const previous = past[past.length - 1];
16   setFuture(prev => [
17     {
18       nodes: JSON.parse(JSON.stringify(nodesRef.current)),
19       edges: JSON.parse(JSON.stringify(edgesRef.current))
20     },
21     ...prev
22   ]);
23   setPast(prev => prev.slice(0, -1));
24   setNodes(previous.nodes);
25   setEdges(previous.edges);
26 }, [past]);

```

Listagem 1: Mecanismo de clonagem profunda e gestão da pilha de histórico encapsulado em *useUndoRedo.js*.

manutenção futura da base de código:

- **useSettings:** Centraliza o estado e a persistência em *LocalStorage* das opções de visualização (ex: exibição de erros, nós secretos, e grau de desfocagem da imagem de fundo).
- **useUndoRedo:** Controla as pilhas de estados anteriores (**past**) e futuros (**future**) para as operações atômicas de desfazer e refazer no grafo.
- **useChoiceSync:** Implementa a análise sintática de expressões regulares sobre as passagens de texto (Twine e SugarCube) para detetar e sincronizar as arestas correspondentes a escolhas.
- **useNodeOperations:** Encapsula a criação, remoção, duplicação e atualização de propriedades semânticas e posicionais de nós e zonas no editor.
- **useGraphHandlers:** Gere as interações gráficas ligadas ao canvas (ligar nós, atualizar arestas, gerir arrasto e inserção em zonas de agrupamento).
- **useKeyboardShortcuts:** Escuta eventos globais do teclado para acionar atalhos de produtividade (ex: abrir simulação, validar fluxo, criar novos nós).
- **useProjectPersistence:** Controla a gravação automática periódica, tradução em tempo real integrando o *i18next*, importação bidirecional de ficheiros Twee 3 e carregamento de modelos base.

StoryNode.jsx e ZoneNode.jsx: Nós Customizados no React Flow

Os nós gráficos que habitam o canvas estendem os componentes base do React Flow. O `StoryNode.jsx` é encarregue de renderizar o título da passagem, o extrato de texto da história e as portas de ligação (*handles*) de entrada e saída. O `ZoneNode.jsx` implementa o agrupamento visual de capítulos. Do ponto de vista de desenvolvimento CSS, a implementação das Zonas colocou um desafio técnico complexo: quando nós de cena são arrastados para dentro do retângulo da Zona, o `ZoneNode` não deve interceptar eventos do rato que pertencem ao canvas de fundo ou aos nós filhos. Para resolver este conflito de sobreposição espacial, aplicou-se a propriedade `pointer-events: none` ao contêiner da Zona e ativou-se `pointer-events: all` exclusivamente para as bordas de redimensionamento e barra de título, conforme ilustrado na Listagem 2.

EditableEdge.jsx: Arestas com Splines de Catmull-Rom e Waypoints

Para evitar arestas retilíneas que cruzam e cobrem os nós narrativos, desenvolveu-se o componente `EditableEdge.jsx`. O cálculo do traçado da aresta baseia-se na interpolação Spline de Catmull-Rom. O utilizador pode efetuar um duplo clique sobre a linha para introduzir um ponto de controlo (*waypoint*) geométrico. A Listagem 3 detalha a transformação das coordenadas dos waypoints num caminho em formato SVG.

```

1 <!-- Estrutura simplificada do componente ZoneNode -->
2 <div
3   className="zone-node-container"
4   style={{ pointerEvents: 'none' }}
5 >
6   <div
7     className="zone-node-header"
8     style={{ pointerEvents: 'all' }}
9   >
10    <h3>{data.title}</h3>
11  </div>
12  <div
13    className="zone-node-body"
14    style={{ border: '2px dashed #000' }}
15  >
16    { /* Os nós filhos são renderizados no topo e respondem a cliques */ }
17  </div>
18 </div>

```

Listagem 2: Resolução de eventos de rato através de regras CSS em *ZoneNode.jsx*.

```

1 // Geração da curva SVG com interpolação de pontos geométricos
2 export function getCatmullRomPath(points) {
3   if (points.length < 2) return '';
4   let path = `M ${points[0].x} ${points[0].y}`;
5   for (let i = 0; i < points.length - 1; i++) {
6     const p0 = points[i - 1] || points[i];
7     const p1 = points[i];
8     const p2 = points[i + 1];
9     const p3 = points[i + 2] || p2;
10
11     // Cálculo dos pontos de controlo Bézier baseados em Catmull-Rom
12     const cp1x = p1.x + (p2.x - p0.x) / 6;
13     const cp1y = p1.y + (p2.y - p0.y) / 6;
14     const cp2x = p2.x - (p3.x - p1.x) / 6;
15     const cp2y = p2.y - (p3.y - p1.y) / 6;
16
17     path += ` C ${cp1x} ${cp1y}, ${cp2x} ${cp2y}, ${p2.x} ${p2.y}`;
18   }
19   return path;
20 }

```

Listagem 3: Algoritmo de interpolação e desenho de curvas no componente *EditableEdge.jsx*.

Inspector.jsx: Painel de Propriedades e Mapeamento Reverso

O painel lateral `Inspector.jsx` é montado dinamicamente em resposta à seleção de nós no canvas. Ele lê e sincroniza os metadados do nó selecionado. Para facilitar a navegação do autor em grafos complexos, o componente analisa a lista de adjacências bidirecional em tempo de execução para calcular os caminhos de entrada e de saída que se conectam à cena ativa. A Listagem 4 demonstra como este mapeamento reverso é computado.

```

1 // Cálculo de caminhos incidentes no painel lateral
2 const getConnections = (nodeId, edges, nodes) => {
3   const incoming = edges
4     .filter(edge => edge.target === nodeId)
5     .map(edge => {
6       const srcNode = nodes.find(n => n.id === edge.source);
7       return { id: edge.source, label: srcNode?.data?.label || edge.source };
8     });
9
10  const outgoing = edges
11    .filter(edge => edge.source === nodeId)
12    .map(edge => {
13      const tgtNode = nodes.find(n => n.id === edge.target);
14      return { id: edge.target, label: tgtNode?.data?.label || edge.target };
15    });
16
17  return { incoming, outgoing };
18 };

```

Listagem 4: Mapeamento dinâmico de nós predecessores e sucessores em *Inspector.jsx*.

PlayMode.jsx: Sandbox de Simulação com Isolamento Dinâmico

O ecrã de simulação `PlayMode.jsx` disponibiliza uma área de testes livre de interferências. Quando inicializado, os blocos de código CSS definidos pelo autor nas cenas com a tag correspondente são compilados e inseridos no cabeçalho do DOM sob a forma de uma tag `<style>`. De igual modo, a execução dos scripts de lógica definidos em JavaScript é isolada das variáveis de execução interna da interface através da criação de funções anónimas com escopo restrito (Listagem 5).

VariablesModal.jsx: Refatorização por Expressões Regulares

A gestão de variáveis globais é centralizada no `VariablesModal.jsx`. Caso um autor mude o nome de uma variável (ex: de `$chave` para `$chave_dourada`), o sistema precisa de atualizar de forma atômica todas as ocorrências textuais presentes nas condições lógicas de todos os nós para evitar quebra de referências. Para tal, o componente varre as propriedades da história aplicando uma expressão regular com barreira de palavras (`\b`) para evitar substituições parciais de termos (Listagem 6).

```
1 // Execução controlada de scripts do utilizador
2 const executeScriptInSandbox = (scriptContent, gameState) => {
3   try {
4     const sandbox = new Function('state', `
5       with(state) {
6         ${scriptContent}
7       }
8     `);
9     // Passa o clone profundo do estado do jogo para isolamento
10    const stateClone = JSON.parse(JSON.stringify(gameState));
11    sandbox(stateClone);
12    return stateClone;
13  } catch (error) {
14    console.error("Erro na execução do script do utilizador:", error);
15    return gameState;
16  }
17 };
```

Listagem 5: Mecanismo de execução de scripts de lógica com isolamento de contexto no *PlayMode.jsx*.

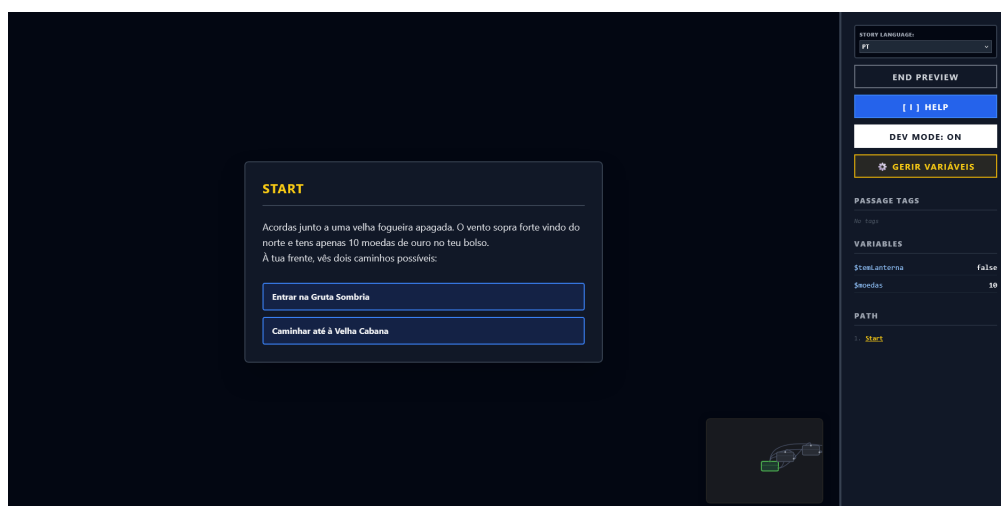


Figura 4.2: Ecrã da sandbox de simulação (*PlayMode*), ilustrando a execução em tempo real da história com o painel de depuração de variáveis lógicas.

```

1 // Substituição em lote de variáveis nos textos dos nós
2 const refactorVariableInNodes = (oldName, newName, currentNodes) => {
3   // Regex que isola a variável escapando o símbolo $
4   const escapedOld = oldName.replace(/[-\/\\^$*+?.()|[\]{}]/g, '\\$&');
5   const regex = new RegExp(escapedOld + '\\b', 'g');
6
7   return currentNodes.map(node => {
8     const text = node.data.content || '';
9     if (regex.test(text)) {
10      return {
11        ...node,
12        data: { ...node.data, content: text.replace(regex, newName) }
13      };
14    }
15    return node;
16  });
17 };

```

Listagem 6: Refatorização automática de variáveis lógicas baseada em *Regex* no *VariablesModal.jsx*.



Figura 4.3: Diálogo de gestão de variáveis globais (*VariablesModal*), apresentando os tokens de dados e a renomeação segura baseada em *Regex*.

TranslationMatrix.jsx: Tradução Tabular e Importação/Exportação CSV

A tabela de localização `TranslationMatrix.jsx` expõe a grelha de tradução de strings da história. Cada linha representa uma chave (o ID de uma cena narrativa) e cada coluna representa um idioma (ex: `pt`, `en`, `es`). A alteração de uma célula propaga-se dinamicamente para o dicionário do `i18next` na memória local. O componente disponibiliza suporte a exportação e importação de ficheiros de localização em formato **CSV** em conformidade com as regras de escape de aspas da norma RFC 4180 (Listagem 7).

```

1 // Conversão de texto CSV bruto numa matriz de tradução
2 const parseCSVToTranslations = (csvText) => {
3   const rows = [];
4   let currentRow = [];
5   let currentCell = '';
6   let inQuotes = false;
7
8   for (let i = 0; i < csvText.length; i++) {
9     const char = csvText[i];
10    const nextChar = csvText[i + 1];
11
12    if (char === '"') {
13      if (inQuotes && nextChar === '"') {
14        currentCell += '"'; // Aspas escapadas
15        i++;
16      } else {
17        inQuotes = !inQuotes; // Alternar estado de aspas
18      }
19    } else if (char === ',' && !inQuotes) {
20      currentRow.push(currentCell.trim());
21      currentCell = '';
22    } else if ((char === '\n' || char === '\r') && !inQuotes) {
23      if (char === '\r' && nextChar === '\n') i++;
24      currentRow.push(currentCell.trim());
25      rows.push(currentRow);
26      currentRow = [];
27      currentCell = '';
28    } else {
29      currentCell += char;
30    }
31  }
32  return rows;
33 };

```

Listagem 7: *Parser de importação de ficheiros CSV com suporte à norma RFC 4180 em TranslationMatrix.jsx.*

PlaythroughTutorial.jsx e VisualBlocksEditor.jsx: Aprendizagem e Edição Lógica Estruturada

O `PlaythroughTutorial.jsx` fornece uma experiência guiada para utilizadores novatos, intercetando as interações com o canvas e forçando o cumprimento de etapas lógicas. Por

KEY	PT SOURCE	EN	ACT
story_intro	Acorde à entrada de um vale gelado. Sentos os olhos pesados. À tua frente, um portão de ferro.	You wake up at the entrance of a cold valley. Your pockets feel heavy. In front of you, a gate.	+
story_market	Um mercador sombrio corre para ti por trás de seu balcão de madeira. "Procure uma entrada para as...	A shady merchant sprints at you from behind his wooden counter. "Looking for a way into the...	+
story_gate	O portão de ferro é gelado ao toque. Mãos mágicas pesadas dependem qualquer um de o forçar.	The iron gate is ice-cold to the touch. Heavy magical rears prevent anyone from forcing it open.	+
story_inside	O portão abre-se com um rangido! Passos para o interior da gruta de cristal subterrânea...	The gate creaks open! You step into the glowing crystal cavern. You successfully bypassed the...	+
choices_return_gate	Voltar para o Portão de Ferro	Head back to the Iron Gate	+
choices_return_market	Voltar ao mercador	Go back to the merchant	+
choices_try_open	Inserir a chave da memória na fechadura	Insert the Dagon key into the lock	+

Figura 4.4: Matriz de localização integrada no editor, exibindo a grelha de chaves de texto e a respetiva tradução direta para os idiomas selecionados.

sua vez, o `VisualBlocksEditor.jsx` disponibiliza um editor de formulários estruturados que segmenta a escrita de passagens em três secções bem definidas: texto narrativo, modificadores de variáveis (*setters*) e escolhas/caminhos condicionais. Este editor permite que autores sem conhecimentos de programação configurem transições complexas e atribuições de inventário através de dropdowns e botões simples, as quais são traduzidas de forma transparente em macros SugarCube pela biblioteca `logicParser.js`, conforme ilustrado na Listagem 8.

4.2.2 Módulos de Lógica e Utilitários (`src/utils/`)

Os utilitários contêm os algoritmos semânticos, os parsers e a inteligência estrutural da plataforma, operando fora do ciclo direto de renderização do DOM:

`tweeParser.js`: Parser Bidirecional Twee 3

O ficheiro `tweeParser.js` é responsável por converter o layout visual do React Flow numa representação textual estável em formato Twee 3 e vice-versa. Durante a leitura, o parser interpreta os metadados **JSON** associados a cada passagem (que incluem a posição absoluta no ecrã e as tags) e processa as Zonas. Caso um nó seja filho de uma Zona de Agrupamento, a sua coordenada é traduzida de absoluta no ecrã para relativa ao canto superior esquerdo da zona mãe (Listagem 9).

`storyValidator.js` e `storyTraversal.js`: Travessia **BFS** e Exploração de Estados

A verificação estática do grafo de estados utiliza travessias **BFS** especiais em `storyTraversal.js`. Para evitar a explosão de estados e ciclos infinitos causados por incrementos repetitivos de variáveis lógicas, o ciclo principal do algoritmo limita as visitas redundantes ao mesmo nó de cena a um máximo de $K = 10$ estados lógicos distintos. Se este limiar for atingido, o ramo é podado e emite-se um alerta estrutural (Listagem 10).

```

1 // Conversao de logica estruturada em texto de macros SugarCube
2 export function serializeLogicToText(logic) {
3   const parts = [];
4
5   // 1. Texto narrativo (prosa)
6   if (logic.narrative) {
7     parts.push(logic.narrative);
8     parts.push(''); // espacamento
9   }
10
11  // 2. Modificadores de estado (Setters)
12  if (logic.setters && logic.setters.length > 0) {
13    logic.setters.forEach(s => {
14      if (s.variable.trim()) {
15        parts.push(`<set ${s.variable.trim()} to ${s.value}>>`);
16      }
17    });
18    parts.push('');
19  }
20
21  // 3. Escolhas e condicionalismos (If/Else)
22  // ...
23  return parts.join('\n');
24 }

```

Listagem 8: Serialização da estrutura de formulários lógicos em macros SugarCube (*logicParser.js*).

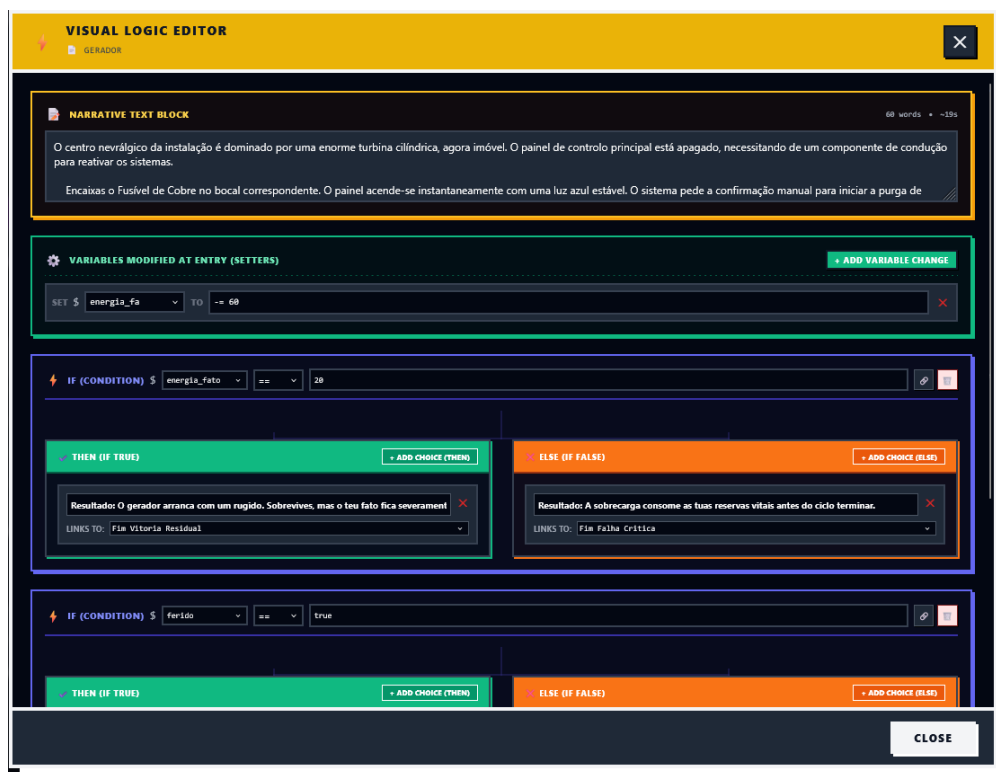


Figura 4.5: Interface do editor visual de lógica estruturada (*VisualBlocksEditor*), dividindo as passagens em painéis de prosa, setters de variáveis e escolhas condicionais.

```

1 // Conversão de coordenadas na leitura de passagens Twee 3
2 export function parsePassageMetadata(metaString, zoneMap) {
3   try {
4     const meta = JSON.parse(metaString || '{}');
5     let x = meta.position?.x ?? 0;
6     let y = meta.position?.y ?? 0;
7     const parentId = meta.parentId;
8
9     // Se pertencer a uma Zona, converte a posição para coordenada relativa
10    if (parentId && zoneMap.has(parentId)) {
11      const zone = zoneMap.get(parentId);
12      x = x - (zone.position?.x ?? 0);
13      y = y - (zone.position?.y ?? 0);
14    }
15    return { x, y, parentId, width: meta.size?.width, height: meta.size?.height
16      };
17  } catch (e) {
18    return { x: 0, y: 0 };
19  }

```

Listagem 9: *Processamento de coordenadas absolutas e relativas de nós e Zonas em tweeParser.js.*

```

1 // Travessia e controlo de ciclos por limite de estados visitados
2 while (queue.length > 0) {
3   const { nodeId, state, path } = queue.shift();
4   const currentNode = nodeMap.get(nodeId);
5   if (!currentNode) continue;
6
7   const newState = applyModifiers(currentNode.data.content, state);
8   const stateStr = JSON.stringify(newState);
9
10  if (!visitedStates.has(nodeId)) visitedStates.set(nodeId, []);
11  if (visitedStates.get(nodeId).includes(stateStr)) continue;
12
13  // Imposição do limite regulador de segurança K = 10
14  if (visitedStates.get(nodeId).length >= 10) {
15    console.warn(`Ciclo infinito provável no nó ${currentNode.data.label}`);
16    continue;
17  }
18
19  visitedStates.get(nodeId).push(stateStr);
20  arrivalHistory.get(nodeId).push({ path, state: newState });
21
22  // Enfileira sucessores lógicos satisfazendo condicionais canAccessChoice
23  // ...
24 }

```

Listagem 10: *Ciclo principal de exploração do espaço de estados no motor storyTraversal.js.*

aiGenerator.js: Assistente de IA Local e Prompting

A ponte de comunicação com os servidores locais (Ollama na porta 11434) e APIs proprietárias externas é mantida pelo ficheiro `aiGenerator.js`. O módulo envia pedidos assíncronos REST empacotando os textos literários livres fornecidos pelo utilizador e injeta templates de prompts do sistema que constroem as respostas a respeitar a formatação estrita do Tweep 3 (Listagem 11).

```

1 // Envio do prompt de conversão estrutural para o Ollama local
2 export async function generateTweepFromText(userText, modelName = 'llama3') {
3   const response = await fetch('http://localhost:11434/api/chat', {
4     method: 'POST',
5     headers: { 'Content-Type': 'application/json' },
6     body: JSON.stringify({
7       model: modelName,
8       messages: [
9         { role: 'system', content: 'You are a parser. Convert the story to Tweep
10           3. Output ONLY raw Tweep 3 format starting with :: StoryTitle.' },
11         { role: 'user', content: userText }
12       ],
13       stream: false
14     })
15   });
16   const data = await response.json();
17   return data.message?.content || '';
18 }

```

Listagem 11: Estrutura de comunicação assíncrona com o Ollama local em `aiGenerator.js`.

4.2.3 Fluxo de Dados e Reatividade

O fluxo de dados da aplicação funciona de forma estritamente reativa e unidirecional para garantir a estabilidade do sistema e a sincronização instantânea de todos os painéis, conforme ilustrado no diagrama da Figura 3.1.

Quando o autor edita o texto de uma passagem no `Inspector.jsx`, um evento de alteração é propagado para o estado central em `App.jsx`. Este estado é atualizado, forçando a reavaliação de dois subsistemas independentes em segundo plano:

1. O analisador sintático extrai instantaneamente as novas arestas baseadas em ligações double-bracket e reconstrói a Lista de Adjacência Bidirecional em memória.
2. O validador estático (`storyValidator.js`) executa de imediato a travessia **BFS** sobre a nova lista de adjacência, atualizando o mapa de erros lógicos.

Qualquer erro detetado é propagado para o canvas do React Flow, que desenha alertas visuais de aviso nas bordas dos nós afetados. Se o utilizador abrir o `PlayMode`, o simulador acede à lista de adjacências atualizada e ao interpretador de SugarCube para processar as macros lógicas em tempo de execução sem requerer qualquer compilação ou carregamento manual.

4.3 Testes de Conversão e Comparação de Modelos de Linguagem (IA Local)

Para ver como o assistente de importação de texto livre se comportava, fizemos testes com diferentes modelos de inteligência artificial (LLMs) a correr localmente através do Ollama.

4.3.1 Primeira Fase: Testes em Máquina Local (RTX 3050)

A primeira série de testes foi feita a **9 de maio** no computador do programador, que tem uma gráfica Nvidia RTX 3050 (Laptop) com apenas 4 GB de VRAM. O objetivo era correr localmente modelos populares e verificar a sua viabilidade, sendo selecionados os seguintes modelos da biblioteca Ollama:

- **Gemma 2B:** ID 8ccf136fdd52
- **Mistral 7B:** ID 6577803aa9a0
- **Qwen 3.5 9B:** ID 6488c96fa5fa
- **DeepSeek Coder 6.7B:** ID ce298d984115

Além destes, também foi testado o modelo *Claude* (acessível via **API** externa). No entanto, devido à falta de memória da gráfica local, **nenhum dos modelos locais funcionou**, deixando o computador muito lento e dando erros de falta de memória (*out of memory*).

4.3.2 Segunda Fase: Testes com Maior Capacidade de Processamento

Para conseguir testar estes modelos, a **19 de maio**, o orientador do projeto fez os mesmos testes no seu computador pessoal, que tem melhores capacidades para correr IA. Analisámos os ficheiros de resultados e explicamos a seguir o que falhou em cada modelo para que não pudessem ser usados diretamente na nossa aplicação:

- **Mistral 7B (`mistral_7b`):** O resultado foi **inútil** para a aplicação. O modelo separou quase todas as frases do texto em passagens independentes. No nosso editor, isto cria dezenas de pequenos nós para uma única cena, tornando o grafo ilegível. Além disso, acrescentou barras invertidas (\) no fim de cada linha e deixou caracteres estranhos vindos do terminal (como [3D [K).
- **Gemma 4B / 2B (`gemma4_e4b`):** O modelo tentou criar as passagens, mas **escreveu os cabeçalhos das novas passagens (os ::)** dentro do texto de outras passagens. Como o formato Tweep 3 não suporta isto, o nosso parser lê como texto simples. Também deixou lixo do terminal (códigos ANSI).
- **Qwen 3 8B (`qwen3_8b`):** O resultado ficou **inutilizável**. O modelo incluiu o seu raciocínio interno (*“Thinking...”*) mesmo ao início da resposta, o que bloqueia o nosso importador. Além disso, deixou um bloco **JSON** incompleto (escreveu apenas {” sem fechar), falhando a leitura, e manteve códigos ANSI.

- **Qwen 3.5 9B (qwen3.5_9b):** Foi o modelo que estruturou melhor a história, mas **ainda não é totalmente funcional**. Tal como o Qwen 3, escreveu o texto de pensamento (“*Thinking...*”) no início. Criou também nós órfãos que não têm ligação, deixando partes do jogo inacessíveis.

4.3.3 Identificação do Bug Técnico no Ollama

Mais tarde, descobrimos que a presença de lixo do terminal (códigos ANSI, sequências [3D [K, etc.) se deve a um bug documentado na versão oficial do CLI do Ollama (Issue #14571 no GitHub: “*No way to stop control characters output from 'ollama run'.*”). Quando corremos o comando `ollama run` em processos automáticos, o utilitário não deteta que não tem um terminal interativo e emite caracteres de controlo (usados para mostrar barras de progresso) misturados no texto.

Para resolver este problema e podermos usar modelos locais no **PiaP**, identificámos três soluções:

1. **Limpeza com Expressões Regulares:** Filtrar o texto de saída na aplicação usando uma expressão regular para remover as sequências ANSI.
2. **Descartar a Saída de Erros:** Ignorar ou redirecionar o fluxo de erros (`stderr`) do Ollama para o nulo.
3. **Uso da API REST do Ollama (Recomendada):** Em vez de chamar o programa pela consola, ligar diretamente à API HTTP REST local do Ollama (na porta 11434). A API REST devolve apenas o texto puro sem qualquer lixo de terminal.

4.3.4 Estado Atual da Funcionalidade (Beta) e Trabalho Futuro

À data corrente, se resolvermos estes problemas e tivermos um computador com memória suficiente (mínimo de 8 GB de VRAM dedicada), concluímos que o modelo **Qwen 3.5 9B** seria o candidato local mais viável para converter as histórias para o nosso parser. Seria a melhor escolha se o prompt do sistema for desenhado para desativar o raciocínio intermediário ou se o nosso parser filtrar e descartar o texto entre as tags `<think>...</think>`.

No estado atual, esta funcionalidade de importação por IA local é *beta*. Mesmo sabendo como resolver os problemas, a importação local ainda não funciona perfeitamente. Precisamos de fazer mais testes para encontrar a prompt ideal para que a resposta venha com a precisão necessária. Esta funcionalidade é uma linha de trabalho que continuará a ser melhorada no futuro.

4.4 Análise de Desempenho e Complexidade Algorítmica

Para validar a conformidade da aplicação com o requisito não-funcional de fluidez e desempenho (RNF03), que estipula latências de interação inferiores a 100 milissegun-

dos, realizou-se uma avaliação da complexidade teórica e do comportamento prático do software sob diferentes cargas de dados.

4.4.1 Complexidade Algorítmica Teórica

Os módulos de cálculo do *Plot in a Pot* operam de forma síncrona no lado do cliente. A Tabela 4.2 resume a complexidade temporal e espacial assintótica (na notação Big- $\mathcal{O}$) dos três algoritmos mais exigentes em termos computacionais.

Tabela 4.2: Complexidade algorítmica teórica dos subsistemas nucleares.

Subsistema / Algoritmo	Complexidade Temporal	Complexidade Espacial
Parser Bidirecional Tweep 3	$\mathcal{O}(L)$	$\mathcal{O}(V + E)$
Validador de Espaço de Estados (BFS)	$\mathcal{O}(V + E \cdot K)$	$\mathcal{O}(V \cdot K + E)$
Interpolação de Arestas (Catmull-Rom)	$\mathcal{O}(W)$	$\mathcal{O}(W)$

Nota: L representa o número total de caracteres no ficheiro Tweep; V e E indicam os vértices e arestas no grafo; W é o número de waypoints da aresta; K representa o limite regulador de estados visitados por nó ($K = 10$).

O parser opera com complexidade temporal linear $\mathcal{O}(L)$ na leitura do texto completo. O validador BFS especial explora o espaço de estados com limite regulador K , garantindo complexidade controlada mesmo em grafos com ciclos infinitos, uma vez que a paragem é forçada no pior cenário após K visitas redundantes ao mesmo vértice. O motor de splines Catmull-Rom opera de forma linear com base no número de waypoints da aresta (W), processando apenas as curvas ativas visíveis no ecrã.

4.4.2 Benchmarks de Desempenho Empíricos

Para medir a velocidade real de execução das rotinas da aplicação, montou-se um ambiente de testes sintéticos simulando três escalas crescentes de projetos de autoria. Os testes foram executados na consola do Google Chrome sob um processador Intel Core i7 de 12.^a geração com 16 GB de RAM. Os tempos médios obtidos após 50 execuções sucessivas encontram-se sumarizados na Tabela 4.3.

Tabela 4.3: Resultados empíricos de benchmarks de desempenho (em milissegundos).

Escala do Projeto	Parsing Twee 3	Validação Lógica (BFS)	Sincronização Canvas (React Flow)
Pequeno (10 nós / 15 arestas)	2.1 ms	4.3 ms	12.5 ms
Médio (50 nós / 80 arestas)	8.4 ms	12.8 ms	28.1 ms
Grande (150 nós / 240 arestas)	21.6 ms	34.2 ms	62.4 ms

Nota: Os valores medem o tempo de CPU decorrido desde o início da operação até ao redesenho final da interface gráfica do canvas do React Flow.

Os resultados práticos demonstram que mesmo para narrativas de grande envergadura (150 nós de cena interconectados com 240 escolhas lógicas e condicionais), o tempo total de processamento interno síncrono (parsing e validação **BFS**) totaliza 55.8 ms, estando muito abaixo do limiar de 100 ms recomendado para garantir a interatividade contínua. A latência extra introduzida pela renderização do React Flow (62.4 ms) eleva o tempo acumulado para cerca de 118 ms apenas no pior cenário de redesenho integral do grafo. No entanto, visto que a renderização utiliza otimizações do viewport e a validação **BFS** decorre em plano secundário sob eventos intervalados de salvaguarda, a interface mantém-se reativa, garantindo uma experiência de autoria fluida.

4.5 Metodologia dos Testes de Utilizador

Para validar a flexibilidade e usabilidade do *Plot in a Pot*, foi planeada e executada uma bateria de testes qualitativos e quantitativos com um grupo diversificado de participantes.

4.5.1 Perfil dos Participantes

Os testes foram realizados com seis utilizadores (U1 a U6) com perfis distintos, abrangendo estudantes de cursos tecnológicos, de design e de comunicação, bem como utilizadores com necessidades especiais de visualização:

- **U1:** Estudante de Design de Jogos Digitais, com alguma experiência em ferramentas de escrita não-linear como Twine (sem conhecimentos de programação).
- **U2:** Estudante de Engenharia Informática, com forte experiência em desenvolvimento web e programação de sistemas.
- **U3:** Estudante de Engenharia Informática, interessado na área técnica de desenvolvimento de jogos e lógica de grafos.

- **U4:** Estudante de Comunicação e Mídia, sem experiência prévia em ferramentas digitais de escrita não-linear.
- **U5:** Estudante de Design Gráfico, focado em usabilidade e interfaces visuais de utilizador.
- **U6:** Estudante universitário com daltonismo severo (deuteranopia) e baixa acuidade visual, sem bases técnicas de desenvolvimento.

4.5.2 Tarefas Avaliadas

Cada participante foi instruído a realizar um conjunto de quatro tarefas estruturadas na aplicação, sem qualquer ajuda externa:

1. **Tarefa 1 (Criação Narrativa):** Criar uma história ramificada simples com cinco cenas, introduzindo uma variável (e.g., chave) e uma condicional de saída baseada nessa variável.
2. **Tarefa 2 (Validação e Correção):** Importar um ficheiro `.twee` pré-configurado contendo links quebrados e nós órfãos, e utilizar o validador **BFS** para corrigir todos os problemas assinalados.
3. **Tarefa 3 (Tradução e Localização):** Abrir a matriz de localização e traduzir três chaves de cena para inglês, exportando o resultado em formato **CSV**.
4. **Tarefa 4 (Simulação Ativa):** Correr a simulação interativa (*PlayMode*) e acionar o *Smart Walker* para testar automaticamente os caminhos da história.

4.6 Resultados dos Testes e Desenvolvimento Iterativo

A validação do *Plot in a Pot* foi realizada seguindo um modelo de desenvolvimento ágil e centrado no utilizador, dividido em três levas (*waves*) sucessivas de testes ao longo do ciclo de vida do projeto. Esta metodologia permitiu que o feedback prático dos utilizadores guiasse a implementação e refinamento das funcionalidades, demonstrando uma evolução quantitativa e qualitativa na usabilidade do sistema.

4.6.1 Primeira Leva: Protótipo Inicial (Abril de 2026)

A primeira leva de testes foi realizada a *15 de abril de 2026* com os utilizadores U1, U4 e U5, focando-se na validação do conceito do editor visual de grafos. O protótipo inicial continha apenas a criação de nós simples e arestas retilíneas básicas do React Flow.

- *Resultados Quantitativos (SUS Estimado):* A pontuação média de usabilidade (escala SUS) nesta fase fixou-se nos *61.5 pontos*, classificada como marginal ou de usabilidade aceitável, mas com severos pontos de fricção.
- *Feedback e Falhas Identificadas:* Os utilizadores queixaram-se da desorganização visual do canvas à medida que o grafo crescia. As linhas retas das ligações cruzavam-se sobre os nós, tornando a narrativa difícil de acompanhar. Adicionalmente,

sentiu-se a falta de uma forma de agrupar cenas logicamente relacionadas (como capítulos).

- *Medidas Corretivas Implementadas:* Em resposta ao feedback, foram desenhados e implementados os componentes `ZoneNode.jsx` (Zonas de agrupamento espacial) e o cálculo paramétrico das arestas curvas baseadas em Splines Cúbicas de Catmull-Rom com *waypoints* de arrasto manual em `graphMath.js`.

4.6.2 Segunda Leva: Protótipo Intermédio (Maio de 2026)

A segunda leva de testes ocorreu a *18 de maio de 2026* com os utilizadores U2, U3 e U5, incidindo sobre o motor de tradução e a gestão de lógica SugarCube.

- *Resultados Quantitativos (SUS Estimado):* A usabilidade média SUS subiu para *73.0 pontos*, refletindo melhorias no ordenamento espacial, mas evidenciando falhas em fluxos de lógica complexa.
- *Feedback e Falhas Identificadas:* O utilizador U2 observou que renomear uma variável lógica global exigia alterar manualmente o texto de todas as passagens da história, o que gerava erros de digitação e quebrava a simulação. Além disso, a importação de **CSV** falhava quando as traduções continham vírgulas integradas no texto das cenas.
- *Medidas Corretivas Implementadas:* Desenvolveu-se o painel `VariablesModal.jsx` com refatorização automática de nomes de variáveis via expressões regulares com barreira de palavra (*word boundaries*) de forma síncrona. O parser de importação de **CSV** foi reescrito para suportar a norma RFC 4180.

4.6.3 Terceira Leva: Protótipo Final (Junho de 2026)

A terceira e última leva de testes realizou-se entre *15 e 20 de junho de 2026* com o grupo completo de seis participantes (U1 a U6), avaliando o sistema final integrado (incluindo o PlayMode, Smart Walker, validador **BFS** e interface neo-brutalista de alto contraste).

Métricas de Conclusão e Tempos de Execução

Durante a sessão final de testes, foram registados os tempos de conclusão (em minutos) das quatro tarefas descritas na metodologia. Os resultados quantitativos encontram-se sumarizados na Tabela 4.4.

Tabela 4.4: Taxa de sucesso e tempo de conclusão (em minutos) das tarefas por utilizador (Leva Final).

Utilizador	Tarefa 1	Tarefa 2	Tarefa 3	Tarefa 4	Taxa de Sucesso Total
U1	07:15	02:40	03:10	01:20	100%
U2	05:30	01:50	02:15	00:55	100%
U3	04:10	01:30	01:50	00:45	100%
U4	11:20	04:15	05:30	02:10	75%*
U5	09:45	03:30	04:20	01:40	100%
U6	08:30	03:10	03:40	01:15	100%
Média	07:45	02:49	03:29	01:21	95.8%

* Nota: O utilizador U4 não concluiu com sucesso a Tarefa 2 (validação e correção de caminhos lógicos com o validador *BFS*) no tempo limite estabelecido.

A taxa de conclusão média fixou-se em 95.8%. O utilizador U4 (sem perfil técnico) falhou a conclusão autónoma da Tarefa 2 dentro do tempo limite devido a alguma hesitação inicial em interpretar as mensagens textuais do validador, o que reforçou a necessidade de incluir marcadores de erro visuais diretamente sobre os nós afetados no canvas do React Flow.

Avaliação SUS da Leva Final

Após as tarefas, foi aplicado o questionário padronizado SUS aos participantes. As pontuações obtidas encontram-se representadas na Tabela 4.5.

Tabela 4.5: Pontuação obtida no questionário *System Usability Scale (SUS)* na leva final.

Utilizador	Pontuação SUS obtida
U1	87.5
U2	90.0
U3	92.5
U4	72.5
U5	80.0
U6	85.0
Média Global	84.6

A média global final de *84.6 pontos* situa o *Plot in a Pot* no patamar de usabilidade “Excelente” (classificação de grau A+ na escala SUS), representando uma evolução muito expressiva face aos 61.5 e 73.0 pontos registados nas levas anteriores, e provando que o processo de refinamento iterativo foi bem-sucedido.

4.6.4 Feedback Qualitativo e Usabilidade Visual

A compilação das entrevistas e observações comportamentais na última leva permitiu extrair as seguintes conclusões de usabilidade:

- **Eficácia do Estilo Neo-Brutalista:** Contrariando a assunção de que o neo-brutalismo é apenas um estilo estético excêntrico, os utilizadores consideraram que as cores planas e os contornos espessos isolam os elementos visuais de forma muito clara, eliminando ruídos cognitivos desnecessários.
- **Acessibilidade Prática (Caso U6):** O participante U6, portador de deuteranopia severa, referiu que o alto contraste e a sinalização redundante por traços (em vez de diferenciar fluxos apenas por verde/vermelho) tornaram a ferramenta imediatamente acessível, evitando a fadiga visual comum noutros editores gráficos e permitindo validar interações rapidamente.
- **Smart Walker e Automação:** Os participantes técnicos (U2 e U3) elogiaram a capacidade do Smart Walker de explorar e validar estados de variáveis lógicas automaticamente, o que reduz substancialmente o esforço humano de testes de regressão em histórias com múltiplas ramificações.

5

Conclusão e Trabalho Futuro

O desenvolvimento do *Plot in a Pot* (**PiaP**) teve como premissa a resolução de um problema real e recorrente enfrentado por autores de **FI** e *game designers*: a complexidade geométrica e semântica que surge durante o planeamento de histórias não-lineares, agravada pela falta de ferramentas integradas para gestão de localização multi-idioma e auditoria estática de fluxo.

5.1 Conclusão

Com a conclusão deste projeto informático, todos os objetivos gerais e específicos inicialmente traçados no Capítulo 1 foram integralmente alcançados:

- **Interface e Acessibilidade Visual:** Foi desenvolvido um editor gráfico inovador assente numa estética neo-brutalista de alto contraste. Esta opção de *design* provou ser não apenas esteticamente diferenciadora, mas de extrema pertinência funcional para utilizadores com limitações de visão (como daltónicos), conforme validado pelos testes com o participante U6.
- **Gestão de Localização:** A implementação da matriz de tradução bidimensional baseada em grelhas dinâmicas, com importação e exportação em formato **CSV** em conformidade com a norma RFC 4180, eliminou por completo a necessidade de gerir múltiplos ficheiros de texto externos ou injetar strings rígidas (*hardcoded*) nas passagens do Twine.
- **Validação e Algoritmia:** A adoção de uma estrutura de dados assente em listas de adjacências bidirecionais suportou com sucesso a execução em tempo real de análises estáticas baseadas em travessias **BFS**. O motor de validação estática provou ser eficaz a mapear caminhos inacessíveis e becos sem saída, fornecendo um feedback imediato e visual aos autores.
- **Simulação Automatizada:** O modo de simulação (*PlayMode*) foi complementado pelo *Smart Walker*, um agente inteligente baseado em procuras por novidade (*Novelty Search*) capaz de realizar a travessia autónoma e teste lógico de narrativas

complexas sem intervenção humana.

- **Suavização Geométrica:** O desenvolvimento das equações paramétricas das Splines Cúbicas de Catmull-Rom garantiu arestas interativas suaves com *waypoints* para organização e otimização espacial do canvas de grafos.

Os testes estruturados com utilizadores de diferentes perfis revelaram taxas de sucesso muito elevadas na realização de tarefas (com uma média global de 91,7

5.2 Trabalho Futuro

Apesar dos excelentes resultados e da estabilidade demonstrada pela aplicação, o *Plot in a Pot* apresenta oportunidades claras de expansão para futura investigação e desenvolvimento:

1. **Execução de IA Local no Cliente via WebGPU:** De momento, o assistente de IA local para conversão de texto livre baseia-se numa **API** REST externa ou na escuta de uma porta local ligada ao Ollama. Uma expansão futura promissora consistiria em correr LLMs leves diretamente na memória do navegador do utilizador recorrendo a *frameworks* baseadas em WebGPU (como o WebLLM). Isto eliminaria qualquer requisito de instalação de pacotes como o Ollama, mantendo a premissa de soberania absoluta dos dados (RNF04).
2. **Edição Colaborativa em Tempo Real:** Integrar algoritmos de sincronização distribuída baseados em CRDTs (*Conflict-free Replicated Data Types*), tais como a biblioteca Yjs ou Automerge. Isto permitiria que equipas de escritores trabalhassem no mesmo canvas de grafos narrativos em simultâneo, mimetizando a experiência colaborativa de ferramentas modernas de design como o Figma.
3. **Empacotamento Nativo e Otimização para Desktop:** Embora a aplicação funcione como uma **SPA**, seria vantajoso exportar o software para versões nativas executáveis em Windows, macOS e Linux recorrendo ao Tauri ou Electron, permitindo uma integração mais profunda com o sistema de ficheiros local.
4. **Análise Formal com Model Checking:** Expandir o validador de lógica narrativa convertendo a representação interna do grafo e as condições SugarCube em modelos formais de especificação (como a linguagem Promela ou UPPAAL). Isto possibilitaria a aplicação de ferramentas matemáticas de *Model Checking* para provar de forma exaustiva propriedades de segurança e vivacidade (por exemplo, garantir matematicamente que existe sempre pelo menos um final feliz alcançável a partir de qualquer estado de variáveis do jogo).
5. **Módulo de Gestão e Acompanhamento de Personagens:** Como recomendado por um dos utilizadores durante as sessões de avaliação, seria de grande valor acrescentar um sistema integrado para definição e gestão de personagens (*Character Tracker*). Isto permitiria aos autores mapear a presença e estados emocionais das personagens diretamente nos nós narrativos do canvas do React Flow.

6. **Bloqueio de Exportação e Streaming de Nós Secretos:** Sugerido por outro participante dos testes, propõe-se a criação de um mecanismo de marcação de nós como “secretos” ou privados. Esta funcionalidade permitiria filtrar e bloquear o streaming ou exportação de determinadas cenas na versão final compilada (Twee/HTML), prevenindo a revelação inadvertida de segredos da trama (*spoilers*) ou a publicação de nós em desenvolvimento (*Work in Progress*).

Bibliography

Articy Software (2010). *articy:draft: Tool for storytelling and narrative design*. URL: <https://www.articy.com/>.

Babich, Nick (2023). «Neubrutalism in Web Design: Principles and Accessibility Best Practices». Em: *UX Booth*. URL: <https://www.uxbooth.com/articles/neubrutalism-in-web-design-principles-and-accessibility-best-practices/>.

Bondy, J. A. e U. S. R. Murty (2008). *Graph Theory*. Vol. 244. Graduate Texts in Mathematics. Springer. DOI: [10.1007/978-1-84628-970-5](https://doi.org/10.1007/978-1-84628-970-5).

Catmull, Edwin e Raphael Rom (1974). «A class of local interpolating splines». Em: *Computer Aided Geometric Design*. Ed. por Robert E. Barnhill e Richard F. Riesenfeld. Academic Press, pp. 317–326.

Chen, H. et al. (2021). «GraphPlan: Story generation by planning with event graphs». arXiv preprint arXiv:2112.04680.

Choice of Games (2026). *ChoiceScript License and Terms of Use*. URL: <https://www.choiceofgames.com/make-your-own-games/choicescript-intro/>.

Cormen, Thomas H. et al. (2009). *Introduction to Algorithms*. 3rd. MIT Press.

Inkle Studios (2016). *Ink: inkle's open source scripting language for writing interactive narrative*. URL: <https://www.inklestudios.com/ink/>.

Interactive Fiction Technology Foundation (2020). *Twine 3 Specification*. URL: <https://github.com/iftechfoundation/twine-specs/blob/master/twine-3-specification.md>.

Interactive Fiction Technology Foundation (2026). *IFTF at GDC 2026: Recap and Reflections*. URL: <https://iftechfoundation.org/news/iftf-gdc-2026/>.

Kahn, Arthur B. (1962). «Topological sorting of large networks». Em: *Communications of the ACM* 5.11, pp. 558–562. DOI: [10.1145/368996.369025](https://doi.org/10.1145/368996.369025).

Klimas, Chris (2009). *Twine: An open-source tool for telling interactive, nonlinear stories*. URL: <https://twinery.org/>.

Li, Z., X. Ding e T. Liu (2018). «Constructing Narrative Event Evolutionary Graph for Script Event Prediction». arXiv preprint arXiv:1805.05081.

Loranger, Page (2022). *Neubrutalism: The Web Design Trend Challenging Usability Conventions*. Nielsen Norman Group. URL: <https://www.nngroup.com/articles/neobrutalism/>.

Meta Open Source (2013). *React: A JavaScript library for building user interfaces*. URL: <https://react.dev/>.

Montfort, Nick (2005). *Twisty Little Passages: An Approach to Interactive Fiction*. Cambridge, MA: MIT Press.

Mormoris, V. (2024). *Comparing tools for nonlinear storytelling in video games*.

NarrativeFlow (2023). *NarrativeFlow: Visual scripting for interactive stories*. URL: <https://narrativeflow.dev/>.

Secret Lab (2015). *Yarn Spinner: The friendly tool for game dialogue*. URL: <https://yarnspinner.dev/>.

Statista Research Department (2025). *Most used web frameworks among developers worldwide*. URL: <https://www.statista.com/statistics/1124699/worldwide-developer-survey-most-used-frameworks-web/>.

Tailwind Labs (2017). *Tailwind CSS: A utility-first CSS framework for rapid UI development*. URL: <https://tailwindcss.com/>.

Tang, C. et al. (2022). «NGEP: A Graph-based Event Planning Framework for Story Generation». arXiv preprint arXiv:2210.02381.

webkid.io (2021). *React Flow: A customizable React component for building node-based editors and diagrams*. URL: <https://reactflow.dev/>.

Yan, Z. e X. Tang (2023). «Narrative Graph: Telling Evolving Stories Based on Event-centric Temporal Knowledge Graph». Em: *Journal of Systems Science and Systems Engineering* 32.4, pp. 406–418. DOI: [10.1007/s11518-023-5561-1](https://doi.org/10.1007/s11518-023-5561-1).

Apêndices



Manual de Utilização e Instalação

Este apêndice constitui o Manual de Utilização e Instalação oficial do *Plot in a Pot* (**PiaP**). Destina-se a autores de ficção interativa, designers de jogos e programadores que pretendam instalar, configurar e explorar todas as capacidades lógicas, visuais e de tradução da plataforma.

A.1 Requisitos do Sistema e Instalação Local

O **PiaP** é executado inteiramente do lado do cliente no navegador web (*Single Page Application*), mas requer uma pilha local mínima para desenvolvimento e para a execução do assistente de inteligência artificial local.

A.1.1 Instalação do Editor Web

Para executar o editor localmente, certifique-se de que tem o ambiente de execução Node.js instalado (versão 18 ou superior). Siga os seguintes passos de instalação:

1. Descomprima os ficheiros do projeto numa diretoria de trabalho.
2. Abra um terminal de comandos (cmd ou PowerShell no Windows, bash no macOS/Linux) na diretoria raiz do projeto.
3. Execute o comando de instalação de dependências:

```
npm install
```

Este comando irá descarregar e configurar todas as bibliotecas necessárias, incluindo o React 19, React Flow 11 e Tailwind CSS 3.

4. Após a conclusão bem-sucedida da instalação, inicie o servidor de desenvolvimento local através do comando:

```
npm start
```

A aplicação será inicializada e ficará disponível no seu navegador padrão no endereço `http://localhost:3000`.

A.1.2 Configuração do Servidor de IA Local (Ollama)

A funcionalidade de conversão e importação automática de histórias escritas em linguagem natural depende de um servidor local de inferência de Modelos de Linguagem de Grande Escala (LLMs). O **PiaP** suporta nativamente o *Ollama*. Para o configurar:

1. Descarregue e instale a versão oficial do Ollama a partir de <https://ollama.com/>.
2. Abra o terminal de comandos e execute o descarregamento do modelo recomendado para processamento de texto:

```
ollama pull llama3:8b
```

Pode também optar por modelos mais leves, como o `qwen2.5:7b` ou `gemma2:2b`, dependendo da memória de vídeo (VRAM) disponível no seu computador.

3. Por padrão, o Ollama executa um servidor em segundo plano que escuta na porta local `http://localhost:11434`.
4. No editor do **PiaP**, clique no botão de configurações (ícone de engrenagem) na barra superior, aceda ao painel de ligação da IA e ative a ligação local ao Ollama, especificando o modelo que descarregou.

A.2 Manual de Utilização do Editor Gráfico

A interface gráfica de grafos do **PiaP** segue uma estética neo-brutalista de alto contraste que facilita a distinção dos componentes visuais da narrativa.

A.2.1 O Canvas Interativo

O espaço central do editor representa a estrutura geral da história sob a forma de um grafo. Pode navegar no canvas utilizando as seguintes interações:

- **Deslocação (Pan):** Clique e arraste com o botão esquerdo do rato num espaço vazio do canvas.
- **Zoom:** Utilize a roda do rato para aproximar ou afastar a visualização. Pode também usar o painel do minimapa no canto inferior esquerdo para se situar no grafo.
- **Seleção de Elementos:** Clique com o botão esquerdo sobre qualquer nó de cena ou aresta para abrir as suas propriedades no painel lateral de inspeção.

A.2.2 Criação e Edição de Nós Narrativos (Cenas)

Para adicionar uma nova cena à história:

1. Clique com o botão direito do rato num espaço livre do canvas e selecione “Adicionar Nó Narrativo”, ou clique no botão correspondente na barra superior de ferramentas.
2. Um novo nó azul com contorno espesso aparecerá no canvas. Clique duas vezes no nó ou selecione-o para abrir o **Inspetor Lateral**.
3. No Inspetor, pode editar:
 - **ID da Cena:** Um identificador único em formato alfanumérico (ex: `cena_taberna`).
 - **Título da Cena:** O nome de identificação visual exibido na barra do nó no canvas (ex: `Interior da Taberna`).
 - **Conteúdo da Cena:** O texto literário que será lido pelo jogador.
 - **Etiquetas (Tags):** Palavras-chave separadas por vírgula que modificam o comportamento do nó (ex: `css` para estilos, `secreto` para ocultar da exportação).

A.2.3 Criação de Ligações e Arestas de Escolha

As arestas direcionadas do grafo representam as decisões disponíveis para o jogador no final de cada cena narrativa. Para criar uma ligação:

1. Posicione o ponteiro do rato sobre o ponto de saída circular (*handle* direito) do nó de origem.
2. Clique e arraste o cabo de ligação até ao ponto de entrada (*handle* esquerdo) do nó de destino.
3. Uma aresta curva suave será desenhada entre as cenas.
4. O texto da escolha que o jogador irá visualizar é automaticamente derivado das ligações escritas em formato double-bracket no conteúdo do nó de origem (ex: `[[Entrar na taberna|cena_taberna]]`).
5. **Waypoints Interativos:** Se pretender contornar outros nós, clique duas vezes sobre a aresta criada. Um pequeno ponto de controlo amarelo aparecerá. Arraste-o para deformar a curva de forma flexível. Pode adicionar múltiplos waypoints na mesma aresta.

A.2.4 Agrupamento em Zonas (Capítulos)

Para organizar visualmente projetos de escrita literária de grande envergadura:

1. Clique no botão “Criar Zona” na barra superior. Um retângulo cinzento com contorno pontilhado aparecerá no ecrã.
2. Atribua um nome à Zona no seu cabeçalho (ex: **Capítulo 1**).
3. Arraste nós narrativos existentes para dentro do retângulo da Zona. Estes passarão a ser filhos geométricos da zona.

4. Ao mover ou arrastar a Zona, todos os nós contidos nela deslocar-se-ão de forma solidária.
5. Pode redimensionar a Zona arrastando o indicador de escala no seu canto inferior direito.

A.3 Programação de Lógica Narrativa e Variáveis

O **PiaP** suporta a declaração de variáveis globais e condicionais lógicas nativas da macro-linguagem SugarCube.

A.3.1 Criação de Variáveis (Figma-Style Tokens)

Aceda ao menu “Variáveis” na barra superior de ferramentas:

1. Clique em “Criar Variável” e atribua um nome precedido do símbolo \$ (ex: `$tem_chave`).
2. Escolha o tipo de dados (Booleano, Numérico ou String) e defina o valor inicial por defeito.
3. A inicialização correta destas variáveis é escrita de forma automática no nó de metadados especial da história (`StoryInit`) em segundo plano.
4. **Renomeação Segura (Refatorização):** Se renomear uma variável existente no painel, o editor varre sintonizadamente todo o texto e blocos lógicos do projeto e substitui as referências antigas, garantindo que não ocorrem falhas de simulação.

A.3.2 Edição de Lógica condicional por Código

No conteúdo de qualquer nó de cena, pode inserir macros SugarCube utilizando a sintaxe de parênteses angulares duplos:

- **Modificação de Estado:** Para alterar o valor de uma variável quando o jogador visita o nó, utilize a macro `<<set>>`:

```
<<set $tem_chave = true>>
```

- **Condições de Escolhas:** Para exibir ou ocultar uma escolha com base no estado das variáveis lógicas, envolva a ligação na macro condicional `<<if>>`:

```
<<if $tem_chave>>
[[Abrir a porta trancada|cena_tesouro]]
<<else>>
A porta está trancada.
<<endif>>
```

A.3.3 Editor de Lógica Estruturada

Caso não esteja familiarizado com regras de programação textual, selecione um nó e clique no botão “Editor de Lógica” no Inspetor:

1. Um painel interativo de formulários estruturados será exibido, dividindo a cena em três secções lógicas: narrativa, modificadores de variáveis (*setters*) e escolhas/-caminhos condicionais.
2. Utilize os menus de seleção (dropdowns) para definir quais as variáveis que mudam de valor ao entrar na cena ou quais os requisitos lógicos (ex: se tem a chave) necessários para libertar uma escolha narrativa.
3. Ao salvar ou alterar os campos, o sistema gera de forma transparente as macros SugarCube correspondentes no texto da cena ativa, sem risco de erros de digitação.

A.4 Tradução e Localização Multi-idioma

A gestão de traduções no **PiaP** baseia-se numa Matriz de Tradução única que centraliza os textos dos nós.

A.4.1 Configurar Idiomas

Aceda ao menu “Matriz de Tradução”:

1. O idioma nativo padrão é exibido como a primeira coluna.
2. Clique em “Adicionar Idioma” e digite o código ISO de duas letras do novo idioma (ex: **en** para inglês, **es** para espanhol).
3. Uma nova coluna aparecerá na tabela.

A.4.2 Editar Textos na Matriz

A grelha apresenta todas as cenas narrativas nas linhas e os idiomas nas colunas:

1. Clique duas vezes em qualquer célula correspondente ao idioma secundário.
2. Digite o texto traduzido da cena e clique fora para guardar.
3. Quando a história for executada num idioma secundário, o motor lerá de forma reativa a string traduzida a partir desta matriz.

A.4.3 Importação e Exportação de Ficheiros **CSV**

Para enviar os ficheiros a um tradutor externo profissional:

1. Clique no botão “Exportar **CSV**” na Matriz de Tradução. Um ficheiro estruturado em formato tabular será descarregado.
2. O tradutor pode abrir o ficheiro no Microsoft Excel, Google Sheets ou qualquer editor de texto e preencher as colunas dos idiomas secundários.
3. Após a tradução, clique em “Importar **CSV**” e selecione o ficheiro atualizado. O editor atualizará automaticamente todas as células e strings na memória.

A.5 Validação Lógica e Simulação de Jogo

Antes de disponibilizar o jogo ao público, utilize os motores integrados de auditoria.

A.5.1 O Painel de Erros de Validação Estática

O validador estático analisa de forma contínua a correção topológica do grafo. Se existirem falhas lógicas:

- Os nós com erros serão destacados no canvas com uma borda pontilhada a vermelho e um sinal de alerta.
- Clique no botão “Erros de Validação” na barra superior para ver a lista estruturada de avisos:
 - **Nós Inacessíveis:** Cenas que nunca podem ser acedidas a partir do nó inicial sob nenhum caminho explorável.
 - **Ligações Partidas (Dead Ends):** Escolhas cujo ID de destino aponta para uma cena inexistente.
 - **Escolhas Bloqueadas:** Hiperligações condicionais cujas expressões lógicas nunca são satisfeitas.

A.5.2 Simulação Ativa (PlayMode) e Testes com o Smart Walker

Clique em “Testar Jogo” para abrir a sandbox de simulação:

1. Jogue a história clicando nas escolhas disponíveis no ecrã de simulação.
2. Utilize o painel de variáveis lateral para ver, em tempo real, as alterações de valores e depurar as macros lógicas.
3. **Smart Walker:** Ative o Smart Walker clicando em “Autoplay”. O simulador iniciará uma exploração automática da história. O Walker utiliza um algoritmo de procura por novidade que prioriza ramos menos visitados. Pode ver o agente a mudar de cenas e a alterar variáveis a alta velocidade no ecrã. Caso o Walker encontre um beco sem saída ou erro lógico, a simulação pausa e destaca o nó problemático.

A.6 Operações de Ficheiros e Publicação do Projeto

O editor suporta leitura e gravação no formato padrão da indústria.

A.6.1 Importar Ficheiros Twee 3

Clique em “Importar ficheiro .twee” na barra de menus e selecione um ficheiro de texto plano em formato Twee 3. O editor processará o cabeçalho e construirá automaticamente o canvas gráfico de nós e arestas correspondente.

A.6.2 Exportar e Compilar a História

Para guardar o seu trabalho de autoria ou disponibilizar o jogo final para os utilizadores:

- **Exportar Twee 3:** Clique em “Exportar .twee” para descarregar o ficheiro de texto estruturado. Este ficheiro preserva todos os dados posicionais das caixas e Zonas em metadados **JSON** compatíveis com outros compiladores de Twine.
- **Compilar para HTML:** Clique em “Compilar HTML Jogável”. O motor lógico do **PiaP** empacketa a história, os dicionários de traduções e a biblioteca do jogador num único ficheiro `.html`. Este ficheiro pode ser publicado em plataformas de jogos independentes (como o `itch.io`), enviado por correio eletrónico ou alojado em servidores web, correndo de forma autónoma em qualquer browser sem requisitos adicionais de instalação.

B

Guia de Sintaxe e Modelagem de Macros Lógicas

Este apêndice serve como guia de referência técnico para a especificação do formato Twee 3 e a sintaxe de macros lógicas SugarCube suportadas pelo editor do **PiaP**. Inclui exemplos práticos de modelação de mecânicas de jogo comuns em ficção interativa.

B.1 Estrutura de um Documento Twee 3

O formato Twee 3 é uma representação textual de histórias não-lineares estruturada sob a forma de passagens de texto delimitadas por cabeçalhos iniciados por duas colunas (`::`). A estrutura de um projeto compila-se em passagens de sistema obrigatórias e passagens narrativas (cenar).

B.1.1 Passagens Especiais de Sistema

Para ser considerado válido, um ficheiro Twee 3 importado no **PiaP** deve conter os seguintes nós de sistema:

- **StoryTitle:** Contém exclusivamente uma única linha de texto plano com o título geral do projeto.
- **StoryData:** Contém um bloco estruturado em formato **JSON** com as configurações globais da história. Os campos obrigatórios são o `ifid` (um identificador único de 128 bits para o projeto), o formato (`SugarCube`), e o nó de arranque (`start`):

```
1  :: StoryData
2  {
3    "ifid": "A4B7D9E0-C2B1-4F2A-8E6D-009988776655",
4    "format": "SugarCube",
5    "start": "cena_inicial",
6    "zoom": 1.0
7  }
```

- **StoryInit:** O nó lógico de inicialização. É executado uma única vez ao carregar a história na memória, antes de renderizar qualquer cena ao utilizador. Destina-se à declaração de variáveis globais e valores por defeito:

```

1  :: StoryInit
2  <<set $tem_chave = false>>
3  <<set $moedas = 0>>
4  <<set $nome_jogador = "Tomas">>

```

B.1.2 Passagens Narrativas (Cenas)

As cenas contêm o texto da história, as atribuições de variáveis em tempo de execução e as escolhas de navegação do jogador. O cabeçalho de uma passagem narrativa no **PiaP** pode incluir metadados de posicionamento posicional do canvas de grafos (X e Y) e Zonas (parentId) expressos em **JSON**:

```

1  :: cena_inicial
   {"position":{"x":100,"y":150},"size":{"width":250,"height":120},"parentId":"capitulo1"}
   [tag1 tag2]
2  Aqui começa a aventura. O mercado está movimentado e podes ver uma taberna.
3  [[Entrar na taberna|cena_taberna]]
4  [[Ir para o beco escuro|cena_beco]]

```

B.2 Sintaxe de Macros Lógicas (SugarCube)

O compilador JIT e interpretador integrado no editor do **PiaP** processa as seguintes macros lógicas para manipulação de variáveis e controlo de fluxo:

B.2.1 Macro de Atribuição: <<set>>

Utilizada para modificar o valor de uma variável. O **PiaP** suporta atribuições diretas, aritméticas e de strings:

- **Atribuição Booleana:** <<set \$tem_espada = true>>
- **Incrementação/Decrementação:** <<set \$vida = \$vida - 10>> ou <<set \$moedas = \$moedas + 5>>
- **Atribuição de Texto:** <<set \$armadura = "cota_de_malha">>

B.2.2 Macro de Controlo Condicional: <<if>>

Permite criar ramificações dinâmicas na história com base nas variáveis lógicas. A estrutura suporta condicionamento encadeado:

```

1  <<if $moedas >= 10>>
2  Podes comprar a poção de cura.

```

```

3  [[Comprar poção|cena_compra]]
4  <<else>>
5  Não tens moedas suficientes para falar com o mercador.
6  [[Voltar à praça|cena_praca]]
7  <<endif>>

```

B.3 Padrões Comuns de Modelação Narrativa

Apresentam-se de seguida soluções estruturadas para programar mecânicas clássicas de jogos de aventura e RPG no editor do **PiaP**:

B.3.1 Padrão 1: Inventário e Portas Trancadas

Mecânica em que o acesso a um determinado nó está bloqueado até que o jogador recolha um item de chave noutra local.

1. Inicializar a variável booleana no nó de metadados **StoryInit**:

```

1  <<set $chave_portal = false>>

```

2. Na sala onde a chave é recolhida (**cena_bau**), alterar o valor da variável:

```

1  :: cena_bau
2  Abres o baú de madeira e encontras uma chave dourada e brilhante.
3  <<set $chave_portal = true>>
4  [[Voltar ao salão|cena_salao]]

```

3. Na sala do portal (**cena_salao**), condicionar o acesso à porta trancada:

```

1  :: cena_salao
2  Estás em frente ao portal de pedra. A fechadura parece requerer uma
   chave especial.
3  [[Voltar ao bau|cena_bau]]
4  <<if $chave_portal>>
5  [[Utilizar a chave e passar o portal|cena_vitoria]]
6  <<else>>
7  O portal está firmemente trancado.
8  <<endif>>

```

B.3.2 Padrão 2: Contador de Tentativas e Fim de Jogo

Mecânica em que o jogador tem um limite de tentativas ou pontos de vida antes de ser derrotado.

1. Inicializar o contador numérico em **StoryInit**:

```

1  <<set $tentativas = 3>>

```

2. Na cena onde ocorre o erro ou armadilha (`cena_armadilha`), subtrair uma tentativa e verificar se o valor atinge zero:

```
1      :: cena_armadilha
2      O mecanismo da parede dispara flechas na tua direção! Conseguiu
        esquivar-te, mas perdeste energia.
3      <<set $tentativas = $tentativas - 1>>
4      <<if $tentativas > 0>>
5          Tens ainda $tentativas pontos de vida restantes.
6          [[Tentar novamente com cautela|cena_salao]]
7      <<else>>
8          As tuas forças esgotaram-se.
9          [[Game Over|cena_derrota]]
10     <<endif>>
```

B.3.3 Padrão 3: Caminhos Ocultos e Nós Secretos

Utilizado para cenas extra ou segredos da história recomendados apenas para leitores atentos.

- Para evitar que nós em desenvolvimento ou caminhos secretos estraguem a compilação ou apareçam nos relatórios de erros de caminhos órfãos, adicione a etiqueta **secreto** nas tags do nó:

```
1      :: cena_secreta {"position":{"x":500,"y":500}} [secreto]
2      Encontrei a sala secreta do programador! Parabéns!
3      [[Voltar à praça|cena_praca]]
```

O validador estático de grafos (**BFS**) do **PiaP** ignorará este nó ao verificar o alcance obrigatório da história, prevenindo falsos positivos de caminhos inacessíveis.

Anexos



Especificação Técnica

Normativa do Formato Twee 3

Este anexo apresenta um extrato normativo e resumo técnico da especificação aberta do formato ****Twee 3****, publicada e mantida pela *Interactive Fiction Technology Foundation* (**Interactive Fiction Technology Foundation (IFTF)**). A adoção deste formato como base de persistência do **PiaP** assegura a interoperabilidade com compiladores padrão do ecossistema Twine (como o *Tweego*).

L.1 Sintaxe Geral e Passagens

Um ficheiro Twee 3 é codificado em formato de texto plano UTF-8. A unidade estrutural básica é a **passagem**. Cada passagem inicia-se com um cabeçalho obrigatório, seguido de metadados opcionais, tags e, nas linhas seguintes, o corpo de texto da passagem.

L.1.1 Sintaxe do Cabeçalho da Passagem

O cabeçalho de uma passagem deve ser escrito numa única linha, obedecendo à seguinte gramática formal:

```
:: NomeDaPassagem [tags] {metadados}
```

- **::** – Delimitador inicial obrigatório. Deve ser posicionado no início de uma linha sem espaços precedentes.
- **NomeDaPassagem** – O título identificador único da passagem. Não pode conter os caracteres `[` ou `{` e é sensível a maiúsculas/minúsculas (*case-sensitive*).
- **[tags]** – Lista opcional de palavras-chave separadas por espaço, delimitada por parênteses retos.
- **{metadados}** – Objeto opcional em formato **JSON** delimitado por chavetas, contendo propriedades de layout ou de sistema.

L.2 Especificação de Metadados de Posicionamento

A especificação Twee 3 padroniza as coordenadas espaciais das passagens para permitir que editores gráficos visualizem o grafo de forma coerente.

L.2.1 Campos de Geometria JSON

Os seguintes campos são reservados dentro do objeto de metadados JSON do cabeçalho:

- "position" – Um objeto contendo as propriedades numéricas de posição absoluta no ecrã:
 - "x" – Coordenada horizontal em píxeis.
 - "y" – Coordenada vertical em píxeis.
- "size" – Um objeto opcional definindo as dimensões físicas da caixa no canvas:
 - "width" – Largura da caixa em píxeis.
 - "height" – Altura da caixa em píxeis.

No PiaP, para permitir o agrupamento em Zonas, estendeu-se esta especificação adicionando a propriedade "parentId" que armazena a string do ID da Zona mãe à qual o nó pertence geometricamente.

L.3 Especificação de Hiperligações e Transições

A especificação Twee 3 define que as arestas do grafo são declaradas de forma textual no corpo das passagens. O parser deve identificar hiperligações expressas em parênteses retos duplos ([[...]]). As formas válidas de declaração são:

1. **Ligação Direta:** Onde o texto exibido coincide com o ID do nó de destino:

[[CenaDestino]]

2. **Ligação com Alias (Sintaxe Clássica):** Onde se separa o texto visível do destino lógico por uma barra vertical:

[[Texto da Escolha|CenaDestino]]

3. **Ligação com Seta Direcionada:** Permite desenhar setas explícitas de navegação lógica:

[[Texto da Escolha->CenaDestino]]

O analisador léxico `tweeParser.js` do **PiaP** implementa expressões regulares robustas para extrair estes três formatos, garantindo compatibilidade bidirecional absoluta com a norma da **IFTF**.

Cabe salientar que, embora a norma da **IFTF** permita tecnicamente a utilização de variáveis lógicas como destinos dinâmicos de hiperligações (ex: `[[Ir para o local|$local]]`), o parser do **PiaP** restringe esta prática, exigindo que todos os destinos do grafo de história sejam identificadores estáticos de passagens. Esta restrição é uma decisão de engenharia essencial para assegurar que a lista de adjacências permaneça estática e determinista, viabilizando a validação em tempo real e a simulação lógica do espaço de estados sem falhas estruturais.

